A Project report on

EXPRESSIVE HUMANOID ROBOTIC HEAD

Submitted in partial fulfillment of the requirements
for the award of the degree of

BACHELOR OF TECHNOLOGY

in

COMPUTER SCIENCE AND ENGINEERING
(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

By

P. SAISREE	**(224G1A3392)**
G. SRAVANTH	**(224G1A33A5)**
M. SHAHID BASHA	**(224G1A33A0)**
B. SAI KUMAR	**(234G5A3307)**

Under the Guidance of

Mr. A. Kiran Kumar, MTech.,(Ph.D)

Assistant Professor

Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY
(AUTONOMOUS)
Rotarypuram Village, BK Samudram Mandal, Ananthapuramu-515701

2025-2026

SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY

(AUTONOMOUS)

(Affiliated to JNTUA, Accredited by NAAC with 'A' Grade, Approved by AICTE, New Delhi & Accredited by NBA (CIVIL, EEE, MECH, ECE & CSE)
Rotarypuram Village, BK Samudram Mandal, Ananthapuramu-515701

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

(ARTIFICIAL INTELLIGENCE & MACHINE LEARNING)

Certificate

This is to certify that the project report entitled EXPRESSIVE HUMANOID ROBOTIC HEAD is the bonafide work carried out by **P. Saisree, G. Sravanth, M. Shahid Basha, B. Sai Kumar** bearing Roll Number **224G1A3392, 224G1A33A5, 224G1A33A0, 234G5A3307** in partial fulfilment of the requirements for the award of the degree of **Bachelor of Technology** in **Computer Science and Engineering (Artificial Intelligence & Machine Learning)** during the academic year 2025-2026.

Project Guide

A. Kiran Kumar, M.Tech.,(Ph.D)
Assistant Professor

Head of the Department

Dr. P. Chitralingappa, M.Tech.,Ph.D.
Associate Professor

Date:
Place: Rotarypuram

External Examiner

DECLARATION

We students of **Computer Science and Engineering (AI & ML)**, **SRINIVASA RAMANUJAN INSTITUTE OF TECHNOLOGY(AUTONOMOUS)**, Rotarypuram, hereby declare that the dissertation entitled **EXPRESSIVE HUMANOID ROBOTIC HEAD** embodies the report of our project work carried out by us during IV year under the guidance of **Mr. A. Kiran Kumar** **M.Tech.,(Ph.D)**, Assistant Professor, Department of Computer Science and Engineering (AI & ML), Srinivasa Ramanujan Institute of Technology, and this work has been submitted for the partial fulfillment of the requirements for the award of degree of Bachelor of Technology.

The results embodied in this project report have not been submitted to any other University or Institute for the award of any Degree or Diploma.

Date:

Place:

S.No.	Name of the Student	Roll Number	Signature
1	P. SAISREE	224G1A3392	
2	G. SRAVANTH	224G1A33A0	
3	M. SHAHID BASHA	224G1A33A5	
4	B. SAI KUMAR	234G5A3307	

Vision & Mission of the SRIT

Vision:

To become a premier Educational Institution in India offering the best teaching and learning environment for our students that will enable them to become complete individuals with professional competency, human touch, ethical values, service motto, and a strong sense of responsibility towards environment and society at large.

Mission:

- Continually enhance the quality of physical infrastructure and human resources to evolve in to a center of excellence in engineering education.
- Provide comprehensive learning experiences that are conducive for the students to acquire professional competences, ethical values, life-long learning abilities and understanding of the technology, environment and society.
- Strengthen industry institute interactions to enable the students work on realistic problems and acquire the ability to face the ever changing requirements of the industry.
- Continually enhance the quality of the relationship between students and faculty which is a key to the development of an exciting and rewarding learning environment in the college.

Vision & Mission of the Department of CSE(AI&ML)

Vision:

To evolve as a leading department by offering best comprehensive teaching and learning practices for students to be self-competent technocrats with professional ethics and social responsibilities.

Mission:

DM 1: Continuous enhancement of the teaching-learning practices to gain profound knowledge in theoretical & practical aspects of computer science applications.

DM 2: Administer training on emerging technologies and motivate the students to inculcate self-learning abilities, ethical values and social consciousness to become competent professionals.

DM 3: Perpetual elevation of Industry-Institute interactions to facilitate the students to work on real-time problems to serve the needs of the society.

Program Educational Objectives (PEOs)

An SRIT graduate in Computer Science & Engineering, after three to four years of graduation will:

PEO 1: Lead a successful professional career in IT / ITES industry / Government organizations with ethical values.

PEO 2: Become competent and responsible computer science professional with good communication skills and leadership qualities to respond and contribute significantly for the benefit of society at large.

PEO 3: Engage in life-long learning, acquiring new and relevant professional competencies / higher academic qualifications.

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompany the successful completion of any task would be incomplete without the mention of people who made it possible, whose constant guidance and encouragement crowned our efforts with success. It is a pleasant aspect that we have now the opportunity to express my gratitude for all of them.

It is with immense pleasure that we would like to express my indebted gratitude to our project Guide **Mr. A. Kiran Kumar, M.Tech.,(Ph.D), Assistant Professor, Department of Computer Science and Engineering (AI & ML)**, who has supported us a lot and encouraged us in every step of the project work. we thank him for the stimulating support, constant encouragement and constructive criticism which have made possible to bring out this internship work.

We express our deep-felt gratitude to **Mr. B. Vijaya Bhaskar Reddy, M.Tech., Assistant Professor, Department of Computer Science and Engineering (AI & ML)**, project coordinator for his valuable guidance and unstinting encouragement enabled us to accomplish our project successfully in time.

We are very much thankful to **Dr. P. Chitralingappa M.Tech., Ph.D., Associate Professor & Head of the Department, Computer Science and Engineering (AI & ML),** for his kind support and for providing necessary facilities to carry out the work.

we wish to convey our special thanks to **Dr. G. Balakrishna, Ph.D., Principal of Srinivasa Ramanujan Institute of Technology (Autonomous)** for giving the required information in doing my internship. Not to forget, we thank all other faculty and non-teaching staff, and my friends who had directly or indirectly helped and supported us in completing our internship in time.

we also express our sincere thanks to the Management for providing excellent facilities and support**.**

Finally, we wish to convey my gratitude to our family who fostered all the requirements and facilities that we need.

Project Associates
224G1A3392
224G1A33A5
224G1A33A0
234G5A3307

ABSTRACT

This project presents the design and implementation of an expressive humanoid robotic head capable of real-time emotion recognition and inter active verbal response. The system is developed in Python and integrates computer vision, deep learning, and embedded hardware control. Facial emotion recognition is performed using a Mini-Xception convolutional neural network implemented in PyTorch, while OpenCV and Dlib are employed for face detection, alignment, and eye-blink monitoring.

An Eye Aspect Ratio (EAR)based module detects prolonged eye closure and generates drowsiness alerts using Dlib's 68point facial landmark model. Physical facial expressions are produced using 19 MG996R servo motors controlled by an Arduino Uno through a lightweight serial communication protocol.

Conversational responses are generated using the Groq API with the Llama-3.1 8B Instant model and converted to speech using the Windows Speech Synthesizer, while user input is captured through Google Speech Recognition. The system was evaluated with 210 participants and achieved an overall expression recognition accuracy of 88.97percent. Strain measurements of the silicone eyelid mechanism showed a peak deformation of 0.082, indicating close similarity to human facial movement. The complete system was developed with a total cost below dollar 650, demonstrating a cost-effective and accessible platform for affective human-robot interaction research.

Keywords: Humanoid Robotics, PyTorch, Mini-Xception, Eye Aspect Ratio, FACS, Dlib, OpenCV, Arduino Serial, Groq LLM, Emotion Recognition, Servo Actuation, Speech Recognition, PowerShell TT

PROJECT CONTRIBUTION TOWARDS SUSTAINABLE DEVELOPMENT GOALS

Project Title:
Expressive Humanoid Robotic Head

SDG - Project Mapping:

S.No.	Sustainable Development Goals	Mapped (Yes/No)
1	SDG 1: No Poverty	No
2	SDG 2: Zero Hunger	No
3	SDG 3: Good Health and Well-Being	Yes
4	SDG 4: Quality Education	Yes
5	SDG 5: Gender Equality	No
6	SDG 6: Clean Water and Sanitation	No
7	SDG 7: Affordable and Clean Energy	No
8	SDG 8: Decent Work and Economic Growth	Yes
9	SDG 9: Industry Innovation and Infrastructure	Yes
10	SDG 10: Reduced Inequalities	Yes
11	SDG 11: Sustainable Cities and Communities	No
12	SDG 12: Responsible Consumption and Production	No
13	SDG 13: Climate Action	No
14	SDG 14: Life Below Water	No
15	SDG 15: Life on Land	No
16	SDG 16: Peace Justice and Strong Institutions	No
17	SDG 17: Partnerships for the Goals	Yes

Explanation:

- The project supports healthcare applications by assisting in monitoring emotions and detecting drowsiness, contributing to better mental and physical well-being.
- It enhances education and learning by providing an interactive AI-based robotic system that improves student engagement and understanding.
- The system promotes innovation in robotics and AI, integrating computer vision, deep learning, and embedded systems.
- It helps in reducing inequalities by enabling assistive technologies for elderly people, differently-abled individuals, and patients.

Expected Impact:

Parameter	Impact
Human Interaction	Improved natural communication
Healthcare Support	Emotion monitoring & drowsiness detection
Education	Enhanced interactive learning
Innovation	Advancement in AI & Robotics

CONTENTS

List of Figures **X**
List of Tables **XI**
List of Abbreviations **XII**
Chapter 1: INTRODUCTION **1-4**
1.1 Robotic head 1
1.2 Problem Statement 4
1.3 objectives 4
Chapter 2: LITERATURE SURVEY **5-8**
Chapter 3: DESIGN **9-29**
3.1 Existing Systems 9
3.1.1 System Architecture 9
3.1.2 Hardware 10
3.1.3 Disadvantages 10
3.2 Proposed system 10
3.2.1 Methodology 12
3.3 Hardware 14
3.3.1 Arduino Uno 14
3.3.2 Servo Motor 15
3.3.3 NeoPixel LED 16
3.3.4 Speaker 17
3.3.5 LCD Display 17
3.3.6 Jumper Wires 18
3.3.7 Breadboard 18
3.3.8 Power Supply adapter 19
3.4 Software 19
3.4.1 Arduino IDE 19
3.4.2 IDLE Python Environment 22
3.4.3 Python 23
3.5 Functional Requirements 24
3.6 Non- Functional Requirements 24
3.6.1 Portability 24

3.6.2 Speed 24
3.6.3 Security 24
3.7Methodology 25
3.8Cost Estimation 26
3.9Time Estimation 27
3.10System Architecture 27
3.11work Flow 28
Chapter 4: Implementation 30-52
4.1 Software Implementation 30
4.1.1 Installation of Arduino IDE 30
4.1.2 Hardware Code 34
4.2 Hardware Implementation 51
4.2.1 Implementation of Objective 1 51
4.2.2 Implementation of Objective 2 52
Chapter 5: Testing 53-55
5.1 Testing Approach 53
5.2 Features to be tested 53
5.3 Testing tools and Environments 54
5.4 Test Cases 54
5.4.1 Inputs 54
5.4.2 Expected Output 54
5.4.3 Testing Procedure 55
Chapter 6: Results 56-57
Conclusion 58
References 59
Publications related to project
Project CO-PO Mapping
Internship Certificates
Details of Project Team

List of Figures

Fig.No.	Description	Page No.
1.1	Overview of Robotic Head	3
3.1	Architecture of the Existing System	9
3.2	Architecture of the Proposed System	11
3.3	Block Diagram of Methodology	14
3.4	Arduino UNO	15
3.5	Servo Motor	15
3.6	NeoPixel LED	16
3.7	Speakers	17
3.8	LCD Display	18
3.9	Jumper Wires	18
3.10	Arduino IDE Default Window	20
3.11	Arduino IDE Board Setup Procedure	21
3.12	Arduino IDE Loading Blink Sketch	22
3.13	Python Shell Window	23
3.14	Block Diagram	25
3.1	System Architecture	28
4.1	Flow Diagram of First Objective	50
4.2	Flow Diagram of Second Objective	51
6.1	LCD Display Showing System Status	55
6.2	Real Time Facial Emotion Detection	55
6.3	Head Rotation Detection Alert	56
6.4	Complete System Integration	56

LIST OF TABLES

Table No	Table Name	Page No
3.15	Cost Estimation	26
3.16	Time Estimation	27

LIST OF ABBRIVIATIONS

AI	Artificial Intelligence
API	Application Programming Interface
AC	Alternating Current
DC	Direct Current
LCD	LIquid crystal display
LED	light emitting diode
IDE	integrated development environment
OS	operating system
USB	universal serial bus
TTS	Text-To-Speech
EAR	Eye Aspect Ratio
CNN	Convolutional neural network
HRI	Human robot interaction

CHAPTER-1

INTRODUCTION

Robotics and artificial intelligence have rapidly advanced in recent decades, enabling machines to interact with humans in increasingly natural and intelligent ways. One of the most important areas of modern robotics research is the development of humanoid robots that can communicate and interact with humans using expressions, speech, and gestures similar to those of people. Among these systems, expressive humanoid robotic heads represent an important step toward creating robots capable of engaging in meaningful social interaction. These robotic systems aim to mimic human facial expressions, head movements, and emotional responses to enhance communication between humans and machines. The development of such robots lies within the field of Human–Robot Interaction, which focuses on designing robots that can collaborate and communicate effectively with humans.

1.1 Robotic Head

Humans rely heavily on facial expressions, eye movements, and speech to communicate emotions and intentions. Traditional robots often lack these expressive capabilities, making interactions feel mechanical and unnatural. To overcome this limitation, researchers have begun integrating computer vision, artificial intelligence, and embedded control systems to create robots that can interpret human emotions and respond accordingly. By combining these technologies, robots can recognize human facial expressions, analyze behavioral cues, and generate appropriate responses in real time. This allows robots to behave more like social companions rather than simple automated machines.

Recent developments in computer vision have enabled machines to detect and analyze human faces with high accuracy. Libraries such as OpenCV provide tools for real-time image processing, while machine learning frameworks like PyTorch enable the implementation of deep neural networks for emotion recognition. Facial landmark detection methods using dlib allow the system to track facial features such as eyes, mouth, and head orientation. These technologies enable robots to identify emotional states, monitor eye blinking patterns, and estimate head movements of a human user.

Emotion recognition plays a crucial role in creating expressive humanoid robots. Human emotions such as happiness, sadness, anger, and surprise are typically conveyed through facial expressions. Deep learning models trained on facial expression datasets can classify these emotions by analyzing facial features extracted from camera images. One commonly used model for this purpose is the Mini-Xception architecture, which is designed for efficient and accurate emotion classification from grayscale facial images. By identifying the emotional state of a user, the robot can generate appropriate verbal and physical responses that make the interaction more natural and engaging.

In addition to recognizing emotions, expressive humanoid robots also incorporate voice-based communication systems. Speech recognition technologies allow robots to understand spoken commands and questions from users. At the same time, text-to-speech systems enable robots to respond verbally. Modern conversational capabilities are often powered by large language models such as Llama 3.1, which can generate intelligent and context-aware responses. By combining speech recognition with AI-generated dialogue, robots can participate in simple conversations and provide informative or emotionally supportive responses. Another important component of expressive robotic systems is the ability to produce physical expressions through mechanical movements. Servo motors are commonly used to control the motion of robotic components such as the neck, jaw, and eyes. A microcontroller such as the Arduino Uno can be used to manage these actuators and execute commands received from a higher-level AI system. For example, when the robot speaks, the mouth mechanism can move in synchronization with the audio output to simulate natural speech behavior. Similarly, head rotation can be used to react to a user's head movement or to track the user's position.Visual expression is another essential feature of expressive humanoid robotic heads. Many robots incorporate LED-based eyes to simulate emotional states and attention cues. Programmable LEDs controlled using the Adafruit NeoPixel Library can change colors or brightness to represent different emotional states or system conditions. For instance, the robot may display bright white or green eyes when active and responsive, while dim or blinking lights may indicate inactivity or alert conditions. Another significant capability of advanced robotic heads is the ability to monitor eye blink patterns and head pose. Eye blink detection can be used to determine

whether a user is attentive or drowsy. This is typically calculated using the Eye Aspect Ratio (EAR), which measures the relative distances between eye landmarks. When the eye remains closed for a prolonged period, the system can trigger a drowsiness alert or adjust its behavior accordingly. Head pose estimation, on the other hand, allows the robot to understand the orientation of the user's face. By analyzing the position of facial landmarks, the robot can determine whether the user is looking left, right, up, or down, enabling more interactive responses. The integration of all these technologies results in a highly interactive robotic system capable of observing human behavior, interpreting emotional cues, and responding both verbally and physically. Communication between the AI system and the robotic hardware is typically achieved through serial communication protocols. Commands generated by the AI software are transmitted to the microcontroller, which then controls the mechanical components of the robot. This architecture allows the system to process complex tasks such as emotion recognition and language generation on a computer while delegating real-time motor control to the microcontroller. Expressive humanoid robotic heads have numerous potential applications in various fields. In healthcare, they can serve as assistive companions for elderly individuals or patients who require social interaction. In education, such robots can function as interactive teaching assistants that engage students through expressive communication. They can also be used in customer service environments, entertainment, and research laboratories studying human–machine communication. The growing interest in social robotics highlights the importance of developing systems that can interact with humans in intuitive and emotionally aware ways. By integrating computer vision, artificial intelligence, speech technologies, and embedded

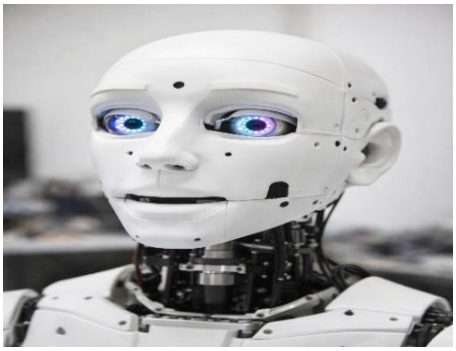


Fig 1.1 Overview of Robotic Head

control systems, expressive humanoid robotic heads represent a promising platform for visual feedback through LED expressions. Overall, this system illustrates how modern AI and robotics technologies can be combined to create robots that are not only functional but also socially interactive.

1.2 Problem Statement

Human interaction with robots should feel natural and responsive, but many low-cost robotic systems cannot understand or react to human emotions and facial movements in real time. Most available robotic head models only perform fixed, pre-programmed actions without intelligent behavior. There is a need for an affordable and practical system that uses Artificial Intelligence to analyze facial expressions and head movements, and then control hardware using the Arduino Uno and servo motors such as MG996R.

The main challenge is to achieve smooth communication between AI-based vision processing and Arduino-controlled mechanical components to create realistic and synchronized human-robot interaction.

1.3 Objectives

To implement real-time facial feature detection using computer vision techniques to track head orientation, eye state (blinking), and jaw movement.

To recognize and classify human emotions such as happiness, sadness, and surprise using machine learning and deep learning models.

To integrate software-based vision processing with hardware actuation through an Arduino microcontroller for real-time mechanical response.

To synchronize detected facial features with physical components, including servo motors for head and jaw movement and LEDs for eye state indication.

To develop a low-cost, open-source, and scalable prototype suitable for educational, research, and assistive robotics applications

CHAPTER-2

LITERATURE SURVEY

Nonverbal cues in human-robot interaction: A communication studies perspective by J. Urakami and K. Seaborn (2019) Human communication relies heavily on nonverbal cues such as gestures, facial expressions, tone of voice, touch, and even smell or taste, which convey emotions, intentions, and social context beyond words. The passage emphasizes that incorporating these cues into human-robot interaction (HRI) can make interactions more natural, engaging, and socially meaningful. By defining nonverbal codes across the five human sensory systems visual, auditory, haptic, olfactory, and gustatory designers can create robots that exhibit behaviors resembling human expressions, enhancing their sense of "aliveness" or social agency. These codes can be translated into actionable design patterns, enabling robots to respond to humans in ways that are intuitive and relatable, while also suggesting research directions to further advance nonverbal communication in HRI.

The integration of nonverbal cues in robots offers several advantages, including enhanced natural interaction, improved communication efficiency, and a stronger social presence that increases user comfort and engagement. Multisensory cues allow robots to interact more immersive, and defined nonverbal codes provide a clear framework for designers. However, there are challenges, such as the complexity and high cost of implementing sophisticated sensors and actuators, cultural differences in interpreting gestures or expressions, and technical limitations in replicating haptic, olfactory, and gustatory cues realistically. Despite these challenges, leveraging nonverbal communication in HRI holds significant potential to make human-robot interactions more lifelike and effective. Expressing robot personality through talking body language by U. Zabala, I. Rodriguez, J. M. Martínez-Otzeta, and E. Lazkano (2013) Social robots need to understand and reproduce the subtleties of human communication to convey messages effectively and build trust with users. Nonverbal cues, such as gestures, body posture, eye movements, and voice intonation, play a crucial role in human interactions, and incorporating them into robots makes interactions more natural and engaging. The system described equips a

humanoid robot with the ability to adapt its body language according to the sentiment of its speech, combining talking-beat gestures with emotional cues like eye lighting, posture, voice intonation, and volume. This approach allows the robot to exhibit a wide range of behaviors and express different personalities, from discrete to more dramatic (histrionic), enhancing the richness and believability of its interactions.

The advantages of this system include improved social presence, more lifelike interactions, and the ability to assign distinct personalities to the robot, making it more relatable and emotionally engaging. By integrating gestures, posture, and expressive cues, the robot can communicate more clearly and generate trust with humans. However, the system also faces limitations, such as the complexity of synchronizing multiple nonverbal cues with speech, the technical challenge of replicating realistic gestures and expressions, and potential cultural differences in interpreting body language. Despite these challenges, the approach demonstrates the potential for creating humanoid robots that not only talk but also express emotions convincingly, offering a more immersive human-robot interaction experience.

Expressing thought: improving robot readability with animation principles by L. Takayama, D. Dooley, and W. Ju (2011)

The passage highlights the importance of animation techniques such as anticipation and reaction in designing human-robot interaction behaviors that are human-readable and predictable. By incorporating forethought before action and expressing reactions to task outcomes (success or failure), robots can communicate their intentions and results more clearly to humans. In the study described, a controlled experiment tested how these behaviors influence people's perceptions of robots and their ability to understand the robot's actions. The results indicated that both showing forethought and reacting to outcomes significantly affect how users perceive the robot's intelligence, intentionality, and reliability, ultimately improving interaction effectiveness and readability.

The advantages of this approach include enhanced predictability, allowing humans to anticipate the robot's actions, which increases user trust and engagement. Forethought and reactions make the robot appear more goal-oriented and socially aware, improving communication without the need for explicit verbal explanations. However, the approach has some limitations: implementing such behaviors requires sophisticated programming

and animation, synchronization with task outcomes, and careful design to avoid misinterpretation of cues. Additionally, responses may vary depending on the context or culture, meaning that behaviors effective in one scenario might not generalize across all interactions. Despite these challenges, integrating anticipation and reaction into robot behaviors is a promising method for creating more intuitive and socially engaging robots**.**

The reliability of non-verbal cues for situated reference resolution and their interplay with language: implications for human robot interaction by S. Gross, B. Krenn, and M. Scheutz, Human communication in task-oriented settings relies heavily on both verbal and non-verbal cues to convey information effectively. When describing objects or actions in a shared environment, humans naturally combine speech with gestures, eye gaze, and body movements to help their interlocutors understand references. The passage emphasizes that for robots to interact naturally in such contexts, they must be able to resolve object references using multi-modal inputs, not just language. Current computational models typically rely on pointing gestures, eye gaze, and object visibility, but research shows that these cues alone only allow resolution of up to 50% of referring expressions. This indicates that additional non-verbal cues are critical for accurate understanding, and understanding their reliability is essential for designing robots with effective natural language understanding in situated tasks.

The advantages of this approach include enabling robots to better understand human intentions in real-world interaction contexts, leading to more accurate object manipulation and task execution. By incorporating a wider range of non-verbal cues, robots can interpret instructions more reliably and respond more naturally, improving human-robot collaboration. However, challenges exist: capturing and interpreting subtle non-verbal cues is technically complex, requires advanced sensing and perception systems, and may vary depending on individual behavior or environmental context. Additionally, combining multiple channels of information can introduce computational complexity and potential ambiguities. Despite these limitations, analyzing and integrating multi-modal cues is crucial for building robots that interact naturally and effectively in collaborative human environments.

Expressive, interactive robots: Tools, techniques, and insights based on collaborations, by J. Gray, G. Hoffman, S. O. Adalgeirsson, M. Berlin, and C. Breazeal (2010)

Creating expressive and interactive robots requires close collaboration between multiple roles, including robot designers, behavior architects, and animators. Each role contributes a unique perspective: animators craft visual and motion content, behavior architects translate these motions into dynamic, context-sensitive behaviors, and robot designers ensure that the physical system can execute these behaviors effectively. The passage highlights lessons learned from such collaborations and introduces tools and techniques designed to facilitate integration while minimizing complexity. These tools aim to give each collaborator flexibility in their role, propagate important constraints across roles, and reduce redundant or automatable work. The focus is on three key areas: sharing animation content with behavior architects, integrating artistic content into dynamic robot behavior, and performing that behavior reliably on the physical robot.

The advantages of this approach include streamlined collaboration, allowing teams to leverage specialized skills without being bogged down by technical constraints, and consistent behavior integration, ensuring that animations, control logic, and physical execution align. It also enables creative flexibility, so animators can focus on expressive content while behavior architects ensure context-aware execution. However, challenges remain: collaboration requires clear communication and coordination, and differences in technical expertise or workflow priorities can create friction. Additionally, integrating artistic content into real-time robot behavior may face hardware limitations or performance constraints, requiring compromises between creativity and feasibility. Despite these challenges, structured collaboration tools and techniques are essential for producing expressive, human-readable, and dynamic robotic behaviors.

CHAPTER-3

DESIGN

3.1 EXISTING SYSTEM

Existing humanoid robotic systems mainly focus on performing predefined actions with limited interaction capabilities. Earlier robotic heads were designed mainly for demonstrations of facial movement and basic communication rather than intelligent emotional interaction. Most of these systems rely on rule-based algorithms and mechanical structures to simulate expressions. Many advanced robots such as Kismet, Sophia, and Ameca use complex mechanical systems with multiple actuators to create realistic facial expressions. These robots integrate computer vision, speech recognition, and motor control systems to simulate human-like communication. However, such systems are expensive and require sophisticated hardware components and powerful computing resources. In most existing systems, facial expressions are generated based on predefined commands rather than real-time emotional understanding. Some systems can detect faces or simple gestures, but they lack complete integration of emotion recognition, speech interaction, and mechanical actuation in a single affordable platform. Because of these limitations, there is a need for a cost-effective robotic system capable of recognizing human emotions and responding through facial movements, speech, and visual feedback.

3.1.1 System Architecture

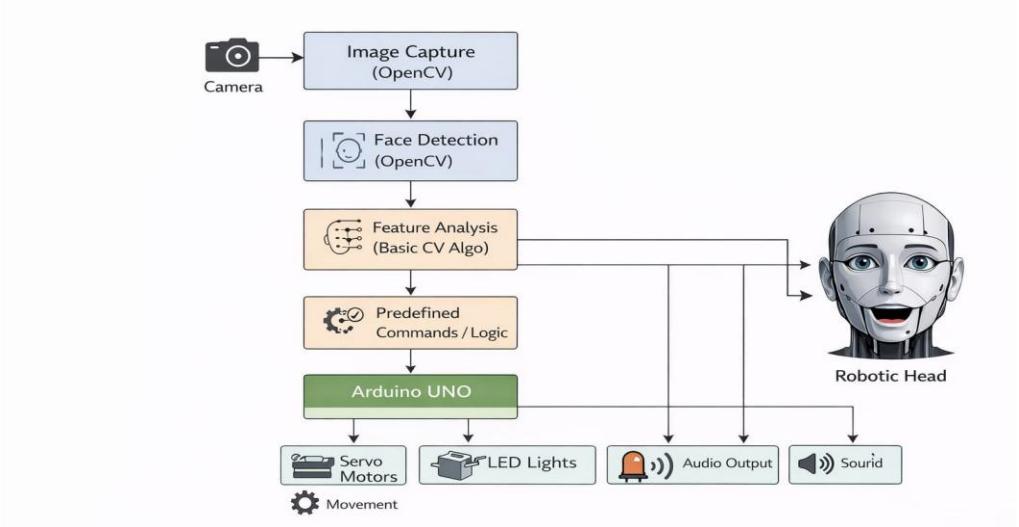


Fig 3.1 Architecture of the Existing System

3.1.2 Hardware:

- Camera
- Arduino UNO
- Servo Motors
- LED Indicators
- Speaker
- Power Supply
- Connecting Wires

3.1.3 Disadvantages:

- Limited Emotion Recognition: Most existing systems only detect faces or simple gestures and cannot accurately recognize human emotions.
- Predefined Responses: The system works based on predefined commands and actions, which limits natural interaction with users.
- Low Intelligence: Traditional systems do not use advanced artificial intelligence or deep learning models for decision making.
- Limited Interaction Capability: Communication between the robot and the user is basic and not fully interactive.
- High Cost in Advanced Systems: Some existing expressive robots require expensive hardware and complex mechanical structures.

3.2 PROPOSED SYSTEMS

The proposed system focuses on the development of an expressive humanoid robotic head capable of interacting naturally with humans through emotion recognition, voice communication, and mechanical expressions. The system integrates computer vision, artificial intelligence, and embedded hardware to create an intelligent robotic platform that can observe human behavior, understand emotional cues, and respond through speech and physical movements. This approach enhances natural communication between humans and robots and contributes to research in Human–Robot Interaction.

The proposed system is designed to operate using a combination of camera-based vision processing, artificial intelligence models, and microcontroller-based hardware control. A camera continuously captures the user's facial expressions and movements. The

captured video frames are processed using computer vision techniques to detect faces, track facial landmarks, and analyze expressions. Emotion recognition is performed using a deep learning model implemented in PyTorch. Based on the detected emotional state, the system generates appropriate responses using an AI language model such as Llama 3.1. These responses are converted into speech using a text-to-speech engine, allowing the robot to communicate with the user in a natural manner.

In addition to verbal communication, the system also controls mechanical components of the robotic head. Servo motors are used to simulate head movement and mouth motion during speech. These motors are controlled through a microcontroller such as the Arduino Uno. The microcontroller receives commands from the AI processing system via serial communication and translates them into physical movements. LED-based eye indicators controlled using the Adafruit Neo Pixel Library are used to provide visual feedback and represent different states of the robot.

The integration of visual perception, emotion recognition, voice interaction, and physical expressions allows the proposed system to create a more engaging and realistic human–robot interaction experience.

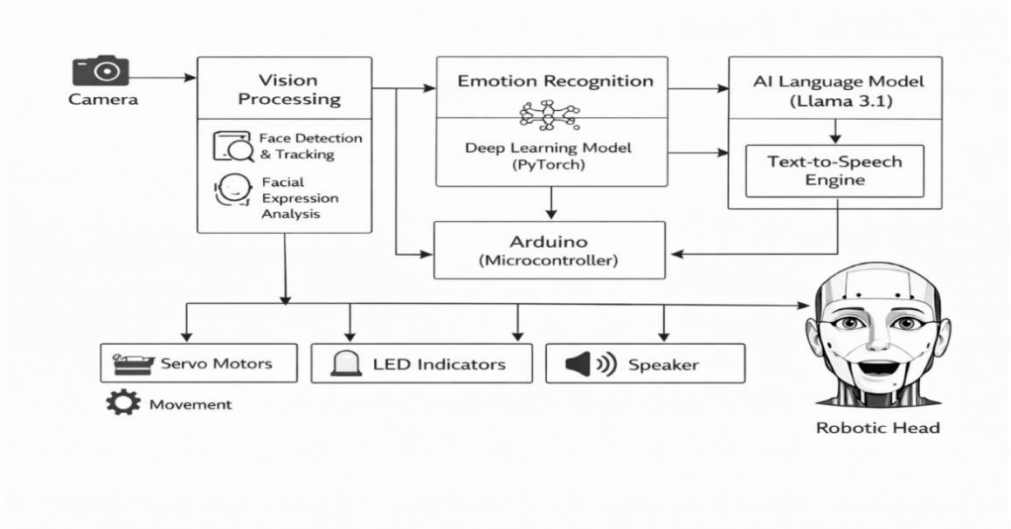


Fig:3.2: Architecture of the proposed system

3.2.1 Methodology

The proposed system follows a multi-stage methodology that combines computer vision, machine learning, and embedded hardware control to enable intelligent robotic behavior.

1. Image Acquisition

The first step of the system involves capturing real-time video from a camera connected to the computer. The camera continuously records frames that are used for facial detection and analysis. Image acquisition is performed using the computer vision library OpenCV. Each frame captured from the camera is converted into grayscale format to simplify the processing and improve computational efficiency.

2. Face Detection

After capturing the video frame, the system identifies the presence of a human face in the image. Face detection is performed using machine learning techniques provided by the dlib or Haar cascade classifiers available in OpenCV. Once a face is detected, the system extracts the region of interest (ROI) containing the face for further analysis.

3. Facial Landmark Detection

The next step involves detecting key facial landmarks such as the eyes, nose, mouth, and jawline. Facial landmark detection identifies specific points on the face that represent important facial features. These landmarks are used to analyze eye blinking patterns and head orientation. By examining the position of these landmarks, the system can determine whether the user is blinking, turning their head, or displaying certain facial expressions.

4. Emotion Recognition

Once the facial region is extracted, the system performs emotion classification using a deep learning model. The extracted facial image is resized to a smaller resolution and normalized before being fed into the neural network. The emotion recognition model used in this system is based on the Mini-Xception architecture. This model processes the input image and classifies it into different emotional categories such as happiness, sadness, anger, surprise, fear, or neutrality.

Emotion detection plays an important role in enabling the robot to understand the user's emotional state and respond accordingly.

5. Head Pose and Eye Blink Detection

The system also calculates head orientation and eye blink patterns. Head pose estimation determines whether the user is looking left, right, up, or down by analyzing facial landmark positions. Eye blink detection is calculated using the Eye Aspect Ratio (EAR) method, which measures the distance between eye landmarks. If the eye remains closed for several frames, the system identifies it as a blink or a sign of drowsiness.

These behavioral cues allow the robot to react dynamically to the user's movements.

6. AI Response Generation

After detecting the user’s emotional state, the system generates an appropriate verbal response. This is achieved using an AI language model such as Llama 3.1. The model processes prompts based on the detected emotion and produces a short conversational response. This step enables the robot to interact naturally with the user and maintain meaningful communication.

7. Speech Recognition and Voice Interaction

To enable two-way communication, the system incorporates speech recognition. Spoken input from the user is captured through a microphone and processed using the Speech Recognition. The recognized text is then sent to the AI model to generate a response. The response is converted into spoken output using a text-to-speech engine such as pyttsx3.

8. Hardware Control

The final step involves controlling the physical components of the robotic head. Commands generated by the AI system are transmitted to the microcontroller through serial communication. The microcontroller interprets these commands and activates the appropriate hardware components.

Servo motors rotate the robot's head or move the mouth during speech.

LED lights change color to represent different emotional states.

An LCD display shows system status messages.

These hardware components allow the robot to express emotions physically and enhance the interaction experience.

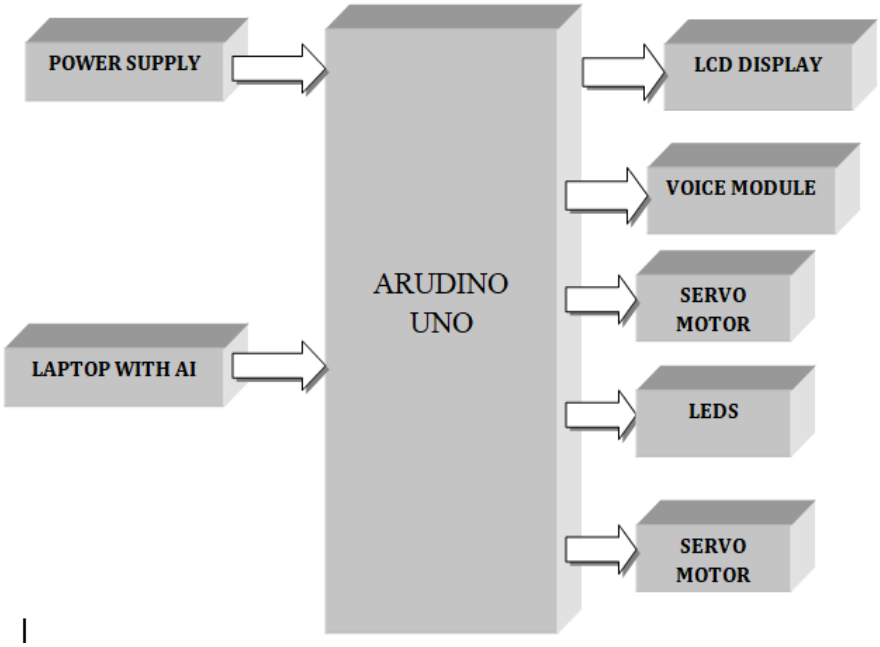


Fig 3.3 Block Diagram

3.3 Hardware

Arduino UNO

Servo Motor

NeoPixel LED

Speaker

LCD (Liquid Crystal Display)

Jumper Wires

Camera Module

Breadboard

Power Supply Adapter

3.3.1 Arduino Uno

The Arduino Uno is the main microcontroller used to control all hardware components in the robotic system. It is based on the ATmega328P microcontroller and is widely used in embedded and robotics projects because of its simplicity and ease of programming.

The Arduino Uno receives control signals from the software system and sends commands to connected devices such as servo motors, LEDs, and the LCD display. It processes these commands and controls the movement and output of the robotic head.

The board provides multiple digital and analog input/output pins which allow communication with various sensors and actuators. It operates on a 5V power supply and can be programmed using the Arduino IDE.

Features:

- ATmega328P Microcontroller
- 14 Digital I/O Pins
- 6 Analog Input Pins
- Operating Voltage: 5V
- USB Interface for programming
- Easy integration with sensors and actuators

The Arduino Uno acts as the **central control unit** of the robotic head s

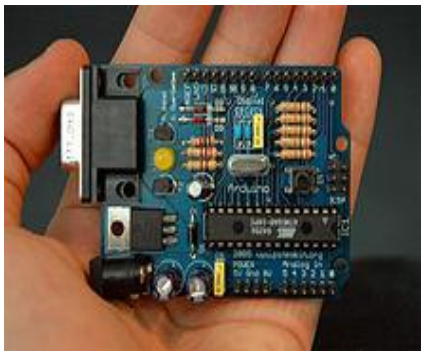


Fig 3.4 Arduino UNO

3.3.2 Servo Motor

Servo motors are versatile components commonly found in robotics, RC vehicles

Fig 3.5d Servo Motor

through feedback mechanisms, ensuring accurate and repeatable motion. By utilizing a closed-loop control system, servo motors can maintain their position even under varying loads or disturbances, enhancing overall system stability and performance.

3.3.3 NeoPixel LED

NeoPixel LEDs are individually addressable RGB LEDs that allow independent control of color and brightness. Each NeoPixel contains an LED and a small controller chip (WS2812 or SK6812) integrated into a single package.

These LEDs can display up to 16 million colors using 24-bit color control (8 bits for Red, Green, and Blue).

One of the major advantages of NeoPixel LEDs is that multiple LEDs can be controlled using a single data pin from a microcontroller.

In this project, NeoPixel LEDs are used as robotic eyes to provide visual expressions.

Functions in the Project:

- Showing robot activity status
- Creating visual expressions
- Simulating emotions

Example LED behavior:

- White light – Robot is active and eyes is opened
- LED off – Robot is idle and eyes is closed

NeoPixel LEDs are controlled using the Adafruit NeoPixel library, which allows easy programming of color patterns and animations.

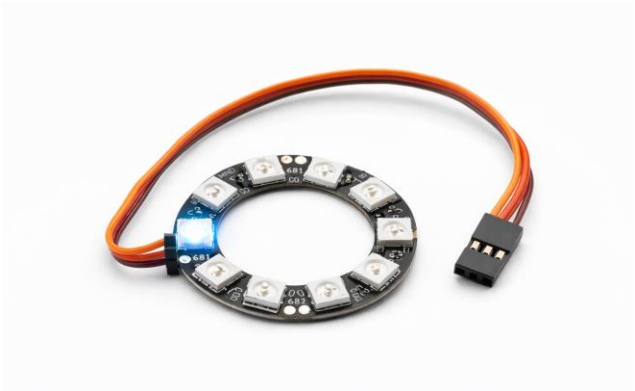


Fig 3.6 NeoPixel LED

3.3.4 Speaker

A Speaker is an electroacoustic device that converts electrical signals into sound waves. It allows the robot to produce voice output and communicate with users.

When an electrical audio signal passes through the voice coil, it produces a magnetic field that moves the diaphragm, creating sound waves. In this project, the speaker is used to generate AI-based voice responses using a text-to-speech system.

Fig 3.7 Speakers

3.3.5 LCD (Liquid Crystal Display)

A Liquid Crystal Display (LCD) is an electronic display module used to show text and information. It is commonly used in embedded systems for displaying system messages and status information.

The LCD used in the project is typically a 16×2 display, meaning it can show:

- 16 characters per line
- 2 lines of text

The display works using liquid crystal molecules placed between two polarizing filters. When voltage is applied, the molecules change alignment, allowing light to pass through and display characters.

The LCD communicates with the microcontroller using either:

- 8-bit mode
- 4-bit mode

In most applications, 4-bit mode is used to reduce the number of required connections.

Functions in the Project:

- Displaying system status
- Showing interaction messages

- Providing visual feedback to the user

Fig 3.8 LCD

3.3.6 Jumper Wires

Jumper wires are simple electrical wires used to establish temporary or permanent electrical connections between different electronic components in a circuit. They are commonly used in prototyping, breadboard circuits, and microcontroller-based projects because they allow easy and flexible connections without soldering.

Fig 3.9 Jumper Wires

3.3.7 Breadboard

A Breadboard is a prototyping device used to build and test electronic circuits without soldering. It consists of a plastic board with many small holes connected internally through metal strips.

Electronic components such as resistors, LEDs, sensors, and jumper wires can be easily inserted into the holes of the breadboard to create temporary circuits. Breadboards are widely used in experimental setups and educational electronics projects.

In the Humanoid Robotic Head project, the breadboard is used to organize and connect electronic components such as LEDs, resistors, and other small modules. It helps in quickly assembling the circuit and making modifications when required.

3.3.8 Power Supply Adapter

A Power Supply Adapter is a device used to provide electrical power to electronic circuits. It converts the AC voltage from the main power source into DC voltage suitable for electronic components.

Most electronic devices such as microcontrollers, sensors, and motors require stable DC voltage to operate properly. Power supply circuits generally include components such as transformers, rectifiers, filters, and voltage regulators.

In the Humanoid Robotic Head project, the power supply provides the required voltage to the Arduino board, servo motors, LEDs, and other electronic modules. Proper power regulation ensures stable operation of the entire robotic system.

3.4 Software

- Arduino IDE
- Python
- IDLE Python Environment

3.4.1Arduino IDE

The Arduino Integrated Development Environment (Arduino IDE) is the software platform used to write, compile, and upload programs to the Arduino microcontroller board. It provides a user-friendly interface that allows developers to create and test embedded programs easily.

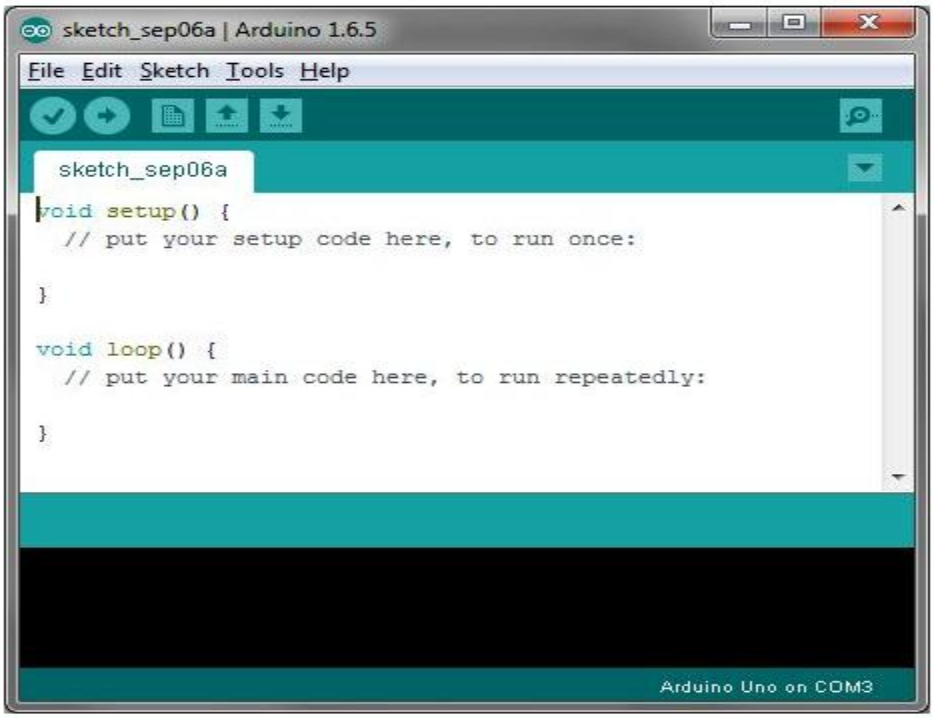
The Arduino IDE consists of several components such as a text editor, message area, console window, toolbar, and menu system. The text editor is used to write the program code, known as a sketch, while the console displays error messages and system information during compilation and uploading.

Programs written in the Arduino IDE are called sketches, and they are saved with the file extension .ino. The IDE provides options for creating new sketches, saving files, compiling programs, and uploading them to the Arduino board..

The IDE also contains several menus such as File, Edit, Sketch, Tools, and Help, which provide additional programming and configuration options.

Arduino IDE Initial Setup

After installing the Arduino IDE, the Arduino board must be connected to the computer using a USB cable. Once the board is connected, the IDE can be used to write and upload programs to the microcontroller.



Arduino IDE Default Window

Fig 3.10 Arduino IDE Default Window

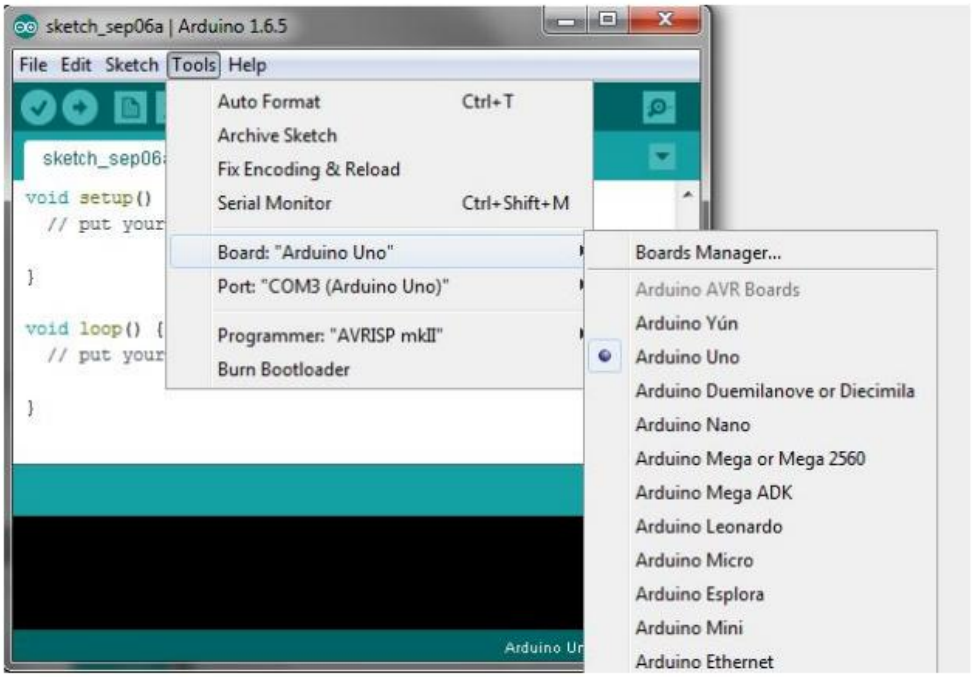
Board Setup in Arduino IDE

To upload programs to the Arduino board, the correct board type must be selected in the IDE.

Steps for board setup:

1. Open the Tools menu in the Arduino IDE.
2. Select Board from the dropdown list.
3. Choose Arduino Uno if you are using the Arduino Uno board.

This step ensures that the program is compiled correctly for the selected hardware.



Arduino IDE: Board Setup Procedure

Fig 3.11Arduino IDE Board Setup Procedure

COM Port Setup

After selecting the board, the correct **COM port** must be configured so that the computer can communicate with the Arduino board.

Steps for COM port setup:

1. Open the **Tools menu**.
2. Select **Port**.
3. Choose the COM port assigned to the Arduino board.

Once the correct port is selected, the board type and COM port number will appear in the **bottom-right corner of the Arduino IDE**.

Testing the Arduino Board

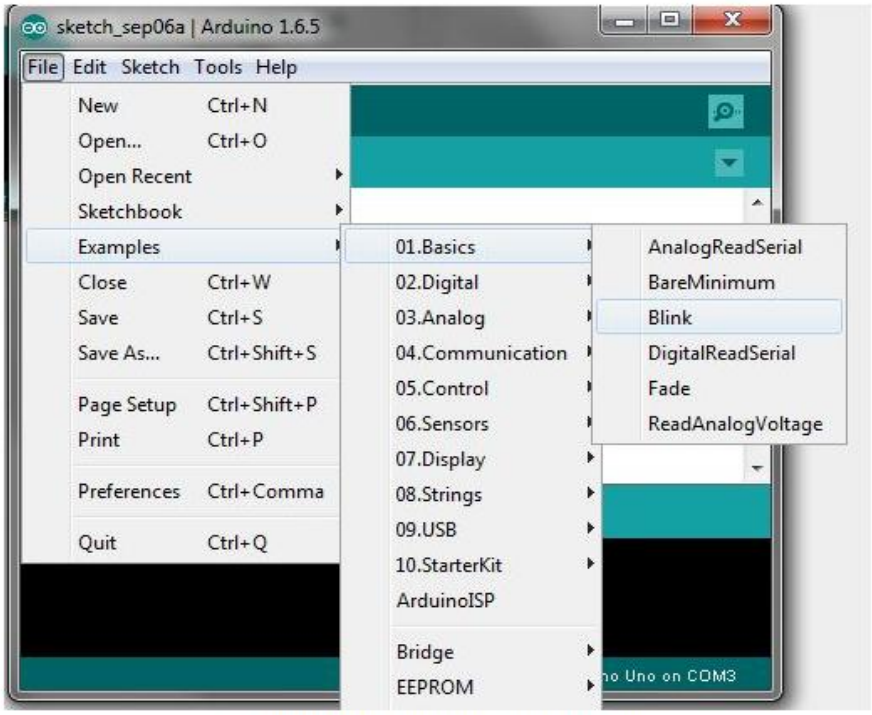
To verify that the Arduino board is correctly connected and configured, a simple test program called **Blink** can be uploaded.

The **Blink program** is included in the Arduino IDE and can be accessed through:

File → Examples → 01. Basics → Blink

This program makes the built-in **LED connected to digital pin 13 blink continuously**. If the LED blinks successfully, it confirms that the Arduino board is working correctly and the program has been uploaded successfully.

Upload Button:



Arduino IDE: Loading Blink Sketch

Fig 3.12 Arduino IDE Loading Blink Sketch

3.4.2Python Programming Using IDLE

IDLE (Integrated Development and Learning Environment) is the standard development environment used for writing and running Python programs. It provides an easy-to-use interface for beginners and developers to develop Python applications.

IDLE includes two main components:

- Python Shell – Used to execute Python commands interactively.
- File Editor – Used to write and edit Python programs.

Python programs are saved with the file extension .py.

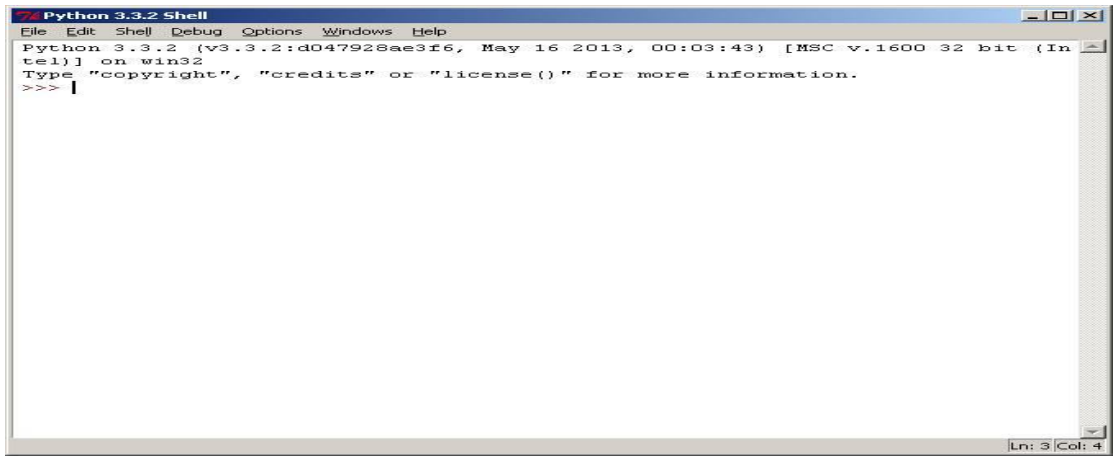


Fig 3.13: Python Shell Window

Steps for Running a Python Program in IDLE

The following steps are used to create and run a Python program using IDLE:

1. Start the IDLE Python environment.
2. The Python Shell window will open.
3. Select File → New Window to create a new Python script.
4. Write the Python program in the editor window.
5. Save the file using File → Save As with the extension .py.
6. Select Run → Run Module to execute the program.
7. The output will appear in the Python Shell window.

3.4.3Python Programming Language

Python is a high-level, general-purpose programming language widely used for software development, artificial intelligence, machine learning, and data processing.

Python is known for its simple syntax, readability, and flexibility, allowing programmers to write programs with fewer lines of code compared to languages such as C++ or Java.

Python supports multiple programming paradigms, including:

- Object-Oriented Programming
- Procedural Programming
- Functional Programming.

It also provides automatic memory management and a large standard library, which makes it suitable for developing complex applications such as emotion recognition systems and computer vision applications.

The standard implementation of Python, known as CPython, is open-source and maintained by the Python Software Foundation.

3.5 Functional Requirements

- The system should be capable of capturing the user's facial image using a camera in real time.
- The system should detect and recognize facial emotions using computer vision and deep learning techniques.
- The system should control the servo motors to simulate human-like movements such as head rotation and mouth movement during interaction.
- The system should display messages or system status on the LCD display to inform the user.
- The system should generate voice responses using a text-to-speech module through the speaker.
- The system should use NeoPixel LEDs as robotic eyes to visually represent different emotional states or system activity.
- The system should establish communication between the Python-based AI program and the Arduino microcontroller to control hardware components.

3.6 Non-Functional Requirements

3.6.1 Portability

The system software developed using Python and Arduino IDE should be compatible with different computer systems and operating systems. The system can be easily deployed and executed on any computer with the required software and libraries installed.

3.6.2 Speed

The system should process camera input and detect facial emotions in real time with minimal delay. It should quickly generate responses such as servo movements, LED expressions, and speech output to maintain smooth human-robot interaction.

3.6.3 Security

The system should ensure that only authorized users can operate the software and access the robotic system. Proper software configuration and controlled access to the hardware components should be maintained to avoid unauthorized usage.

3.7 Methodology

To implement this project, the Iterative Model is used. This model follows a continuous development cycle that includes planning, analysis, design, implementation, and evaluation.

In this project, the development process begins with identifying the system requirements such as facial emotion detection, servo motor control, LED eye expressions, and speech output. During the planning stage, the required hardware components like Arduino, servo motors, speakers, NeoPixel LEDs, LCD display, and camera are selected.

In the next stage, the software components are designed using Python libraries and deep learning frameworks. The system processes facial images through a camera and analyzes facial expressions using a trained emotion recognition model. During the implementation phase, the software communicates with the Arduino microcontroller to control hardware components such as servo motors for robotic head movement, NeoPixel LEDs for eye expressions, and speakers for voice responses. The LCD module displays system messages or interaction status.Testing and evaluation are performed at each stage to ensure that the system correctly detects emotions and responds appropriately through m

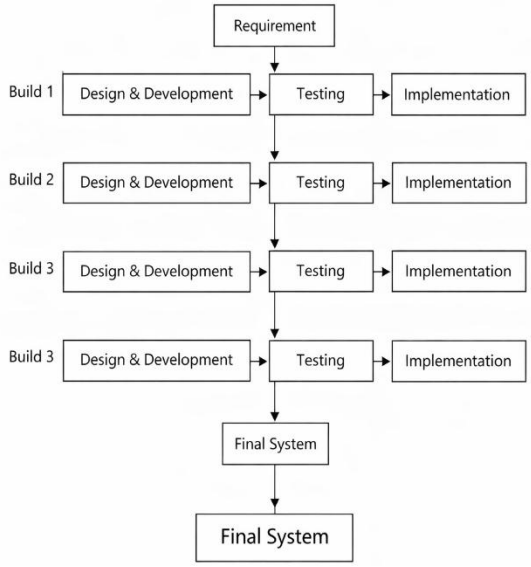


Fig 3.14 :Block Diagram

and voice. Each iteration improves the system performance and interaction capability. The final output of the project is achieved after completing several cycles of development and testing.

Advantages:

- The iterative model allows improvements and modifications at different stages of development.
- Errors and issues can be detected and corrected early during the development process.
- The system can be tested multiple times to improve accuracy and performance.
- It supports continuous development and enhancement of both hardware and software components.
- The model helps in building a reliable and efficient interactive robotic system.

3.8 Cost Estimation

S.NO		Item / Devices	Estimation Cost	Fund required from the Institute in INR total(Rs.)
1.	Non-recurring Items	Arduino Uno Board ATmega328P	740	300
2.		Micro Digital Servo Motor 9G	400	150
3.		LED Lights	160	100
4.		3D Head	6000	3200
5.		Speakers	400	150
6.		LED Panel	150	100
7.		Soldering Machine	550	250
8.	Recurring Items	Consumables (Wires, USB Cable)	150	100
9.		Contingency Expenses(covers, replacement or repairs)	350	150
10.		Travel Expenses	2000	500
		Total Cost	11,250	5,000

Table 3.15 Cost Estimation

3.9 Time Estimation

S.NO	ACTIVITY	DURATION
1.	Gathering all the components	1 week
2.	Developing the Hardware System	4 weeks
3.	Connecting hardware and software	2 weeks
4.	Implementing the system	2 weeks
5.	Rectifying the drawbacks and finalizing the system	1 week

Table 3.16 Time estimation

3.10 System Architecture

The system architecture of the proposed robotic head consists of both software components and hardware components working together to perform emotion recognition and interactive responses.

The system begins with a camera module that captures real-time facial images of the user. These images are processed using computer vision techniques for face detection and facial expression analysis.

The captured facial data is then sent to a deep learning emotion recognition model developed using the PyTorch framework. This model analyzes facial features and classifies the detected emotion such as happiness, sadness, anger, or surprise.

Once the emotion is recognized, the result is processed by an AI language model such as Llama 3.1, which generates an appropriate conversational response. The generated response is converted into speech using a Text-to-Speech engine.

The system then communicates with the Arduino Uno through serial communication. The Arduino microcontroller controls the hardware components of the robotic head.

The hardware components include:

- Servo Motors – used for head movement and mouth movement.
- NeoPixel LEDs – used to simulate expressive robotic eyes.

- Speakers – used to produce voice responses.
- LCD Display – used to display system status or interaction messages.

When the system detects an emotion, the Arduino activates the corresponding servo motor movements, LED eye expressions, and audio responses, allowing the robotic head to interact with the user in a natural and expressive manner.

Thus, the system architecture integrates computer vision, deep learning, artificial intelligence, and embedded hardware to create an interactive humanoid robotic system capable of recognizing and responding to human emotions.

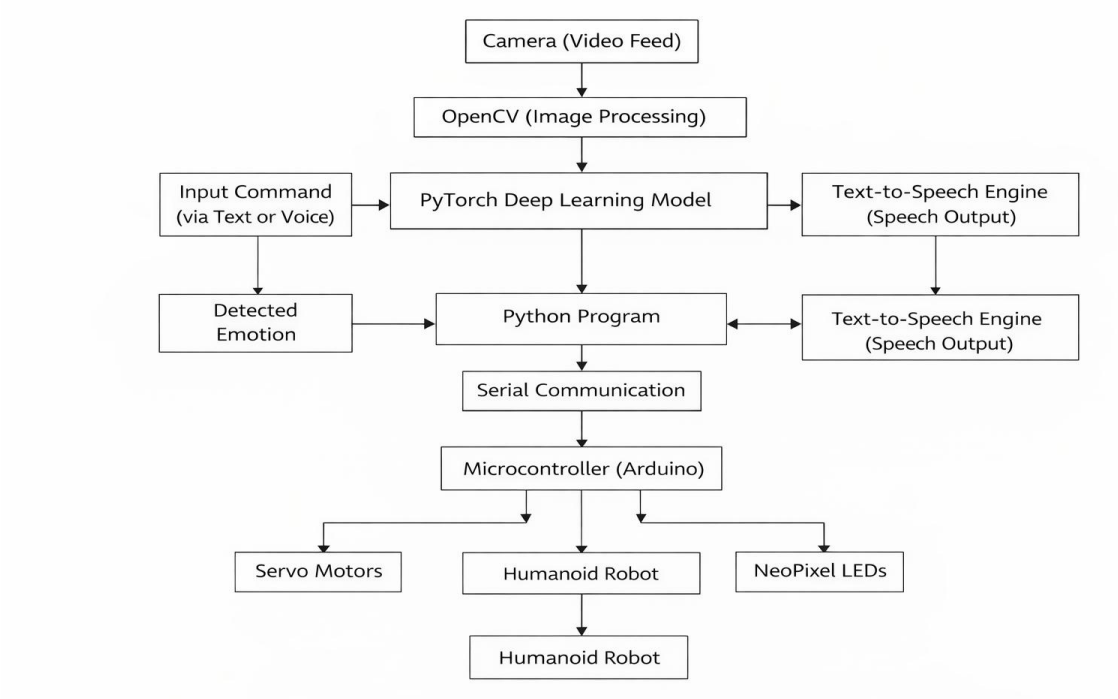


Fig 3.17 System Architecture

3.11 Workflow

The overall workflow of the system follows a continuous cycle of sensing, processing, and responding.

Step 1: Video Capture

A camera captures real-time video frames of the user.

Step 2: Face Detection

The system identifies the user's face in the captured frame.

Step 3: Facial Analysis

Facial landmarks are detected to analyze expressions, eye blinking, and head movement.

Step 4: Emotion Classification

The facial image is processed by the emotion recognition model to determine the user's emotional state.

Step 5: AI Processing

The detected emotion is used to generate an appropriate response using an AI language model.

Step 6: Speech Interaction

The robot speaks the generated response and listens for further user input.

Step 7: Hardware Response

The microcontroller receives commands and controls servo motors, LEDs, and other hardware components to produce physical expressions.

Step 8: Continuous Interaction

CHAPTER 4

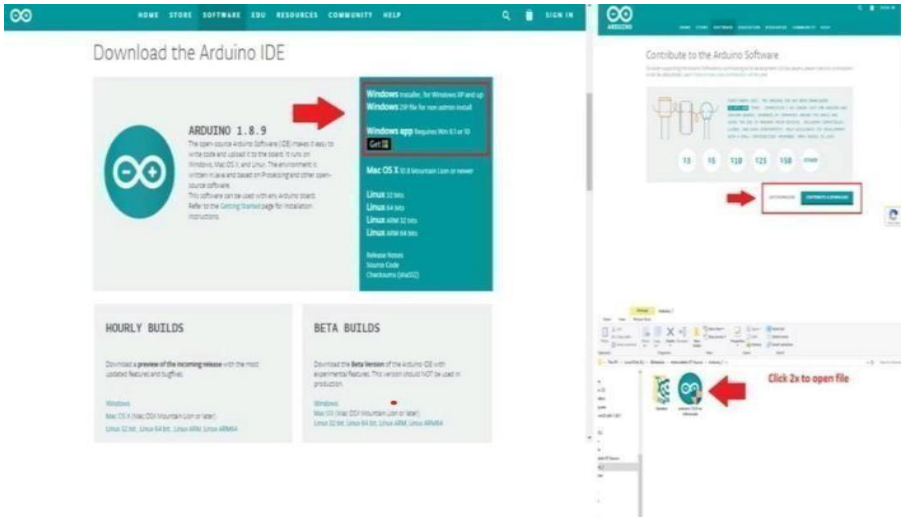
IMPLEMENTATION

4.1 Software implementation

4.1.1 Installation of Arduino Mega IDE.

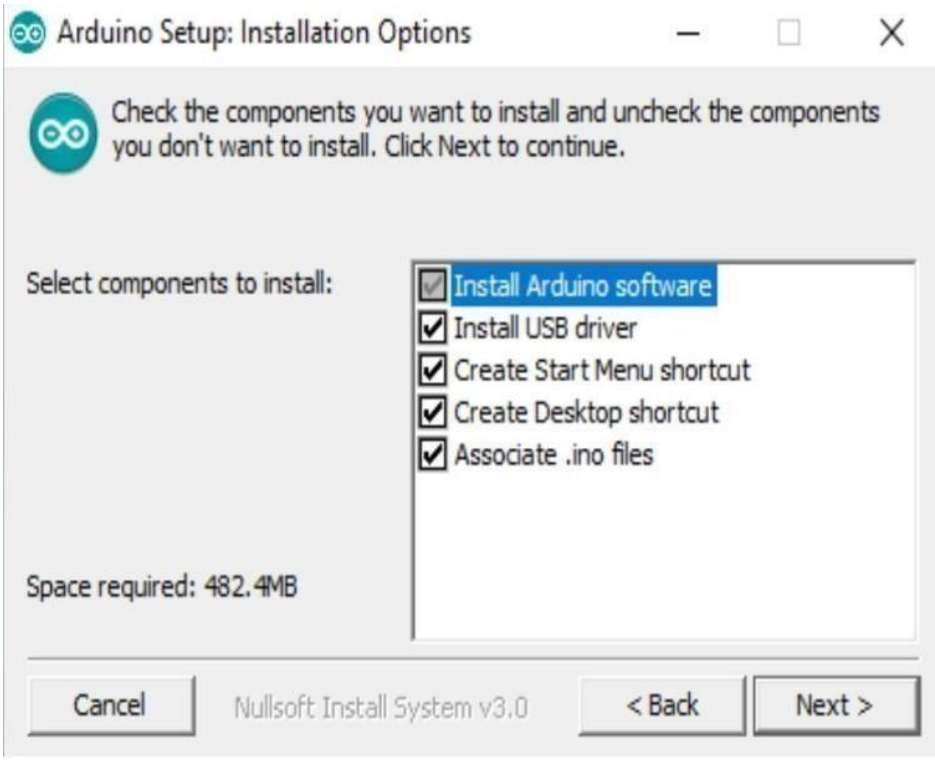
To install Arduino Mega IDE on your Windows PC, follow the next instructions:

Step 1: Download File Arduino IDE

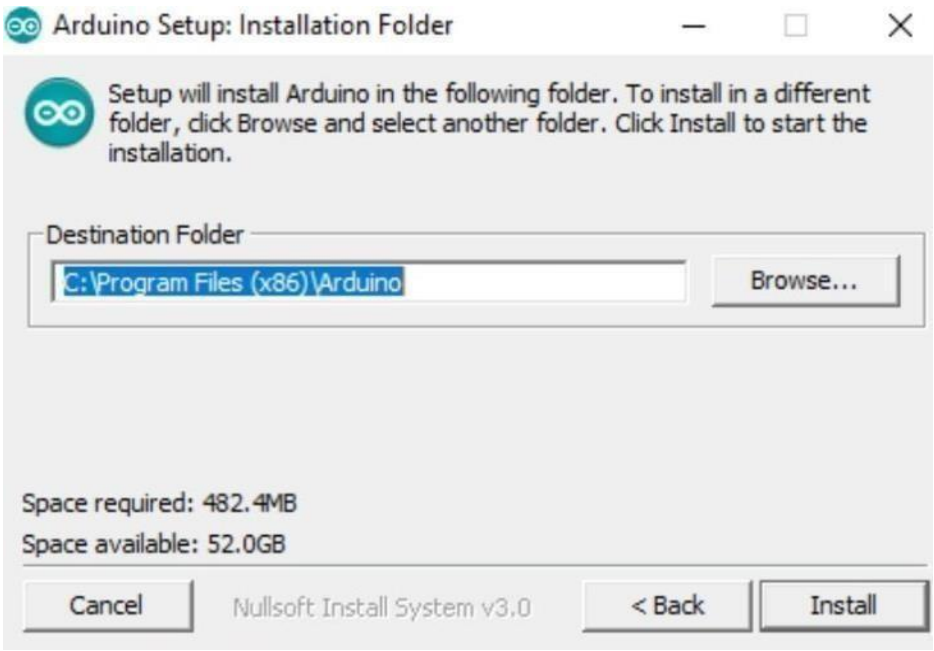


Step 2: License Agreement

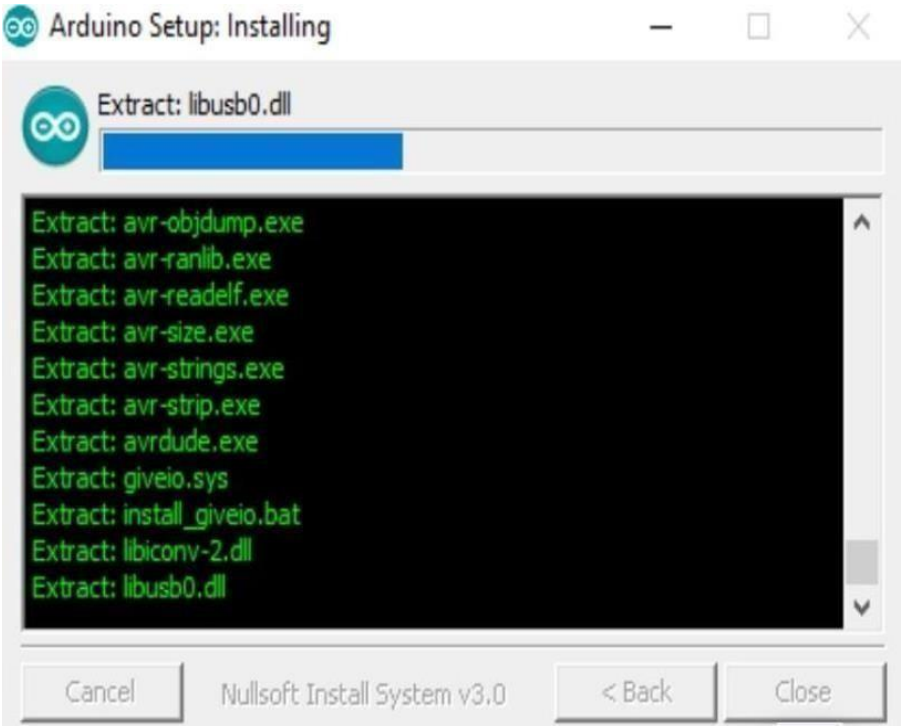
Step 3: Installation Option



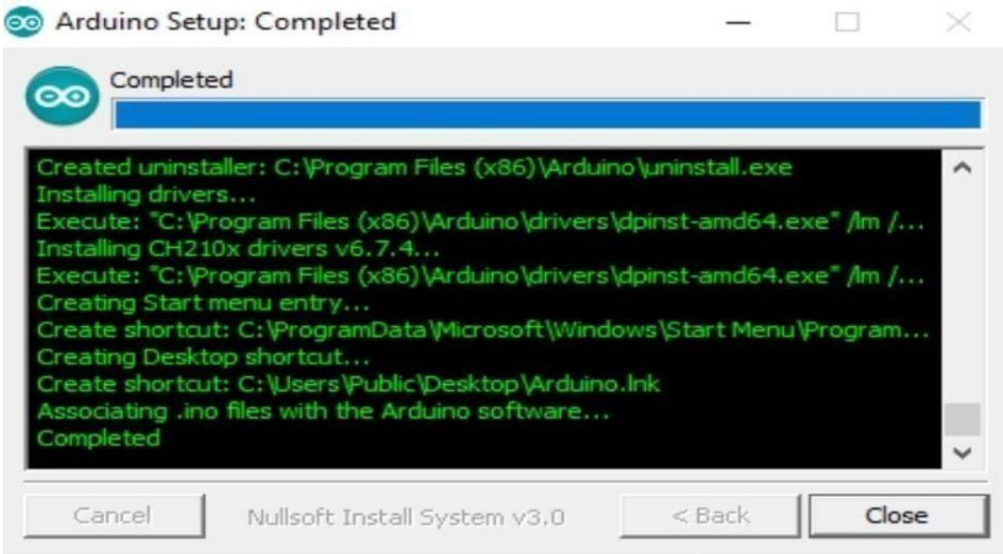
Step 4: Installation Folder



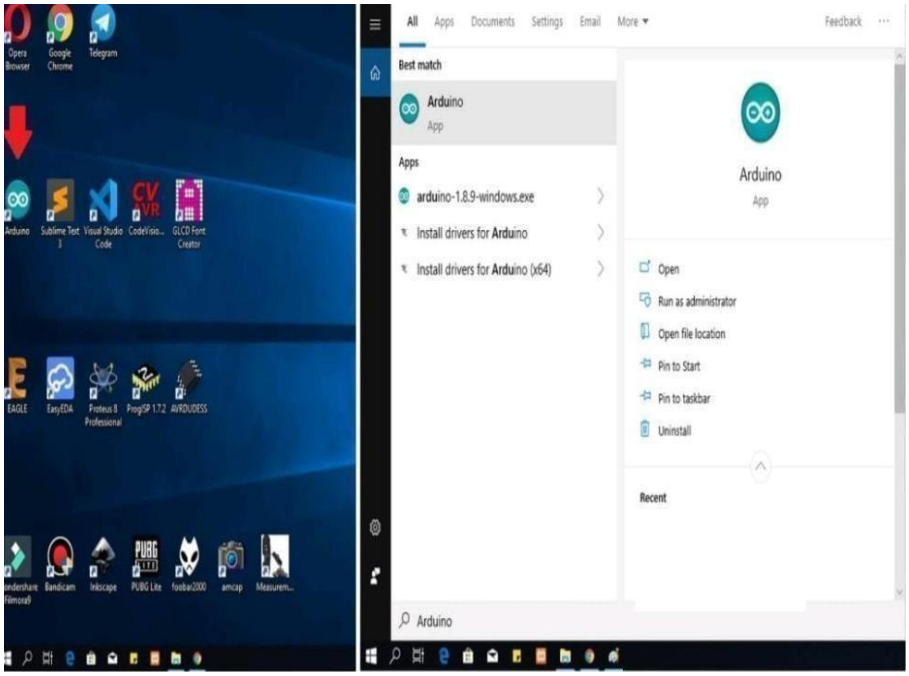
Step 5: Installing Process



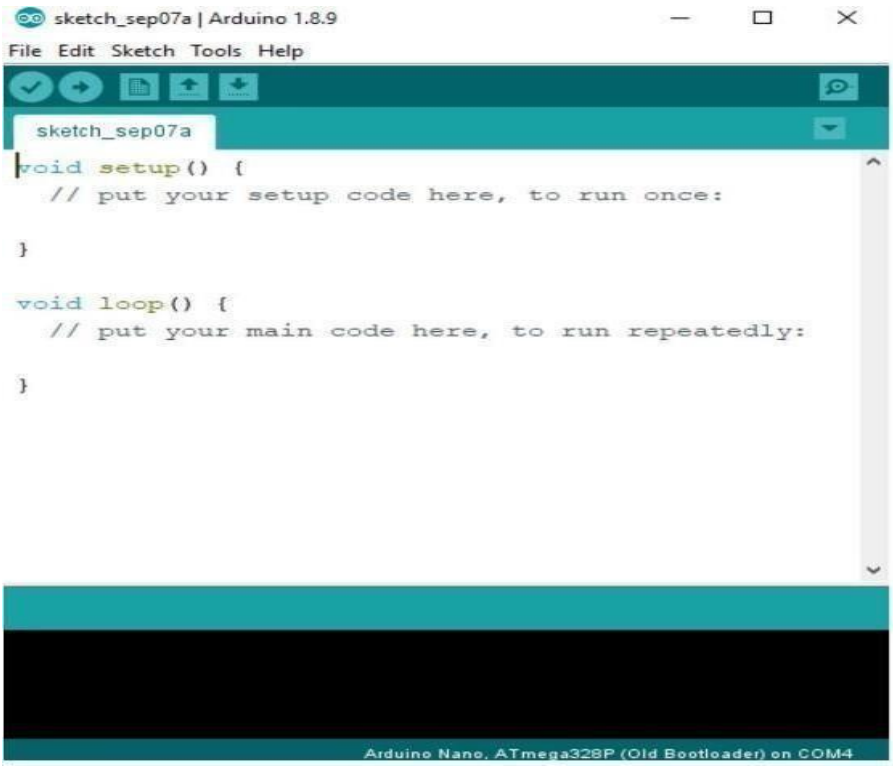
Step 6: Installation Complete



Step 7: Open Arduino IDE



Step 8: Display Arduino IDE



4.1.2 Hardware Code

```
#include <Servo.h>
#include <Adafruit_NeoPixel.h>
#include <LiquidCrystal.h>
// ===== LCD Setup (RS, E, D4, D5, D6, D7) =====
LiquidCrystal lcd(8,9,10,11,12,13);
// ===== Servo Setup =====
Servo servo1;
Servo servo2;
int currentPos1 = 90;
int currentPos2 = 10;
bool servo2Running = false;
bool servo2Direction = true;
// ===== NeoPixel Setup =====
#define LED_PIN1 3
#define LED_PIN2 4
#define NUM_LEDS 1
Adafruit_NeoPixel strip1(NUM_LEDS, LED_PIN1, NEO_GRB + NEO_KHZ800);
Adafruit_NeoPixel strip2(NUM_LEDS, LED_PIN2, NEO_GRB + NEO_KHZ800);
// ===== Smooth Servo Movement =====
void smoothMove(Servo &servo, int &currentPos, int targetPos) {
  if (currentPos < targetPos) {
    for (int pos = currentPos; pos <= targetPos; pos++) {
      servo.write(pos);
      delay(10);
    }
  } else {
    for (int pos = currentPos; pos >= targetPos; pos--) {
```

```
      servo.write(pos);
      delay(10);
    }
  }
  currentPos = targetPos;
}
// ===== LED ON =====
void setWhite() {
  for (int i = 0; i < NUM_LEDS; i++) {
    strip1.setPixelColor(i, strip1.Color(255, 255, 255));
    strip2.setPixelColor(i, strip2.Color(255, 255, 255));
  }
  strip1.show();
  strip2.show();
}
// ===== LED OFF =====
void setOff() {
  for (int i = 0; i < NUM_LEDS; i++) {
    strip1.setPixelColor(i, 0);
    strip2.setPixelColor(i, 0);
  }
  strip1.show();
  strip2.show();
}
// ===== Show Ready =====
void showReady() {
  lcd.clear();
  lcd.setCursor(0,0);
  lcd.print("System Ready");
}
```

```
void setup() {
   Serial.begin(9600);
   servo1.attach(6);
   servo2.attach(7);
    servo1.write(currentPos1);
    servo2.write(currentPos2);
    strip1.begin();
    strip2.begin();
  setOff();
 lcd.begin(16, 2);
   showReady();
}
void loop() {
  if (Serial.available()) {
    char command = Serial.read();
    lcd.clear();
// ===== Command 1 =====
    if (command == '1') {

      lcd.print("NECK ROTATION");
      smoothMove(servo1, currentPos1, 40);
      smoothMove(servo1, currentPos1, 90);
      smoothMove(servo1, currentPos1, 140);
      smoothMove(servo1, currentPos1, 90);

      lcd.setCursor(0,1);
      lcd.print("Done");
      delay(2000);
      showReady();
    }
```

```
// ===== Command 2 =====
    if (command == '2') {
      servo2Running = true;
      lcd.print("TALKING");
      lcd.setCursor(0,1);
      delay(2000);
    }
 // ===== Command 3 =====
    if (command == '3') {
      servo2Running = false;

      smoothMove(servo2, currentPos2, 10);
      lcd.print("Servo2 Stop");
      lcd.setCursor(0,1);
      lcd.print(" STOP TALKING");
      delay(2000);
      showReady();
    }
// ===== Command 4 =====
    if (command == '4') {
      setWhite();
      lcd.print("EYES ON");
      delay(2000);
      showReady();
    }
  // ===== Command 5 =====
    if (command == '5') {
      setOff();
      lcd.print("EYES OFF");
      delay(2000);
```

```
      showReady();
    }
  }
// ===== Continuous Servo2 Movement =====
  if (servo2Running) {

    if (servo2Direction) {
      currentPos2++;
      if (currentPos2 >= 140) servo2Direction = false;
    } else {
      currentPos2--;
      if (currentPos2 <= 10) servo2Direction = true;
    }

    servo2.write(currentPos2);
    delay(10);
  }
}
```

Software code

```
"""
Emotion + Eye Blink + Voice AI Assistant + Serial Signals
Author: Amr Elsersy
Updated: Serial communication + Head Rotation integrated
"""

import sys, os, time, threading, argparse
import cv2
import torch
import numpy as np
```

```
import torchvision.transforms.transforms as transforms
import speech_recognition as sr
import serial
from groq import Groq
from scipy.spatial import distance
from imutils import face_utils
import dlib
import imutils

from face_detector.face_detector import DnnDetector, HaarCascadeDetector
from model.model import Mini_Xception
from utils import get_label_emotion, histogram_equalization
from face_alignment.face_alignment import FaceAlignment

# ---------------- DEVICE ---------------- #
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")

# ---------------- GROQ ---------------- #
client =
Groq(api_key="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")


# ---------------- SERIAL PORT ---------------- #
try:
    ser = serial.Serial('COM3', 9600, timeout=1)
    time.sleep(2)
except:
    ser = None
    print("Warning: Serial port not available")
```

```
# ---------------- WINDOWS TTS ---------------- #
def speak1(text):
    safe_text = text.replace("""", "")
    if ser:
        try:
            ser.write(b'2')
        except:
            pass

    command = f'''powershell -Command "Add-Type -AssemblyName System.Speech; \
    $speak = New-Object System.Speech.Synthesis.SpeechSynthesizer; \
    $speak.Speak('{safe_text}');"""
    os.system(command)

    if ser:
        try:
            ser.write(b'5')
        except:
            pass


import pyttsx3

tts_engine = pyttsx3.init()
tts_engine.setProperty('rate', 170)   # speed
tts_engine.setProperty('volume', 1.0) # volume

def speak(text):
    if ser:
```

```
        try:
            ser.write(b'2')   # speaking start
        except:
            pass

    tts_engine.say(text)
    tts_engine.runAndWait()

    if ser:
        try:
            ser.write(b'3')   # speaking end
        except:
            pass

# ---------------- SPEECH RECOGNITION ---------------- #
recognizer = sr.Recognizer()

def listen():
    with sr.Microphone() as source:
        recognizer.adjust_for_ambient_noise(source, duration=0.5)
        print("Listening...")
        audio = recognizer.listen(source)
    try:
        text = recognizer.recognize_google(audio)
        print("User:", text)
        return text
    except:
        print("Could not understand.")
        return None
```

```
# ---------------- LLM ---------------- #
def ask_llm(prompt: str) -> str:
    response = client.chat.completions.create(
        model="llama-3.1-8b-instant",
        messages=[
            {"role": "system", "content": "Reply in ONE short sentence only."},
            {"role": "user", "content": prompt}
        ],
        temperature=0.6,
        max_tokens=60
    )
    return response.choices[0].message.content.strip()

# ---------------- EYE BLINK ---------------- #
def eye_aspect_ratio(eye):
    A = distance.euclidean(eye[1], eye[5])
    B = distance.euclidean(eye[2], eye[4])
    C = distance.euclidean(eye[0], eye[3])
    return (A + B) / (2.0 * C)

EYE_THRESH = 0.25
EYE_CONSEC_FRAMES = 5
eye_counter = 0

detector_dlib = dlib.get_frontal_face_detector()
predictor = dlib.shape_predictor("shape_predictor_68_face_landmarks.dat")

(lStart, lEnd) = face_utils.FACIAL_LANDMARKS_68_IDXS["left_eye"]
(rStart, rEnd) = face_utils.FACIAL_LANDMARKS_68_IDXS["right_eye"]
```

```
# ---------------- HEAD ROTATION ---------------- #
HEAD_ROTATION_THRESHOLD = 40
head_rotation_sent = False

def get_head_pose(shape, frame_size):
    model_points = np.array([
        (0.0, 0.0, 0.0),
        (0.0, -330.0, -65.0),
        (-225.0, 170.0, -135.0),
        (225.0, 170.0, -135.0),
        (-150.0, -150.0, -125.0),
        (150.0, -150.0, -125.0)
    ])

    image_points = np.array([
        shape[30],
        shape[8],
        shape[36],
        shape[45],
        shape[48],
        shape[54]
    ], dtype="double")

    focal_length = frame_size[1]
    center = (frame_size[1] / 2, frame_size[0] / 2)

    camera_matrix = np.array([
        [focal_length, 0, center[0]],
        [0, focal_length, center[1]],
        [0, 0, 1]
```

```
    ], dtype="double")

    dist_coeffs = np.zeros((4, 1))

    success, rotation_vector, translation_vector = cv2.solvePnP(
        model_points, image_points, camera_matrix, dist_coeffs
    )

    rmat, _ = cv2.Rodrigues(rotation_vector)
    angles, _, _, _, _, _ = cv2.RQDecomp3x3(rmat)

    pitch, yaw, roll = angles
    return pitch, yaw, roll

# ---------------- VOICE THREAD ---------------- #
voice_thread = None
voice_lock = threading.Lock()

def voice_interaction(emotion):
    global voice_thread
    prompt = f"The user looks {emotion}. Reply in ONE short sentence only."
    reply = ask_llm(prompt)
    print("AI:", reply)
    speak(reply)

    user_input = listen()
    if user_input:
        follow_up = ask_llm(user_input)
        print("AI:", follow_up)
        speak(follow_up)
```

```
    with voice_lock:
        voice_thread = None

# ---------------- MAIN ---------------- #
def main(args):
    global eye_counter, voice_thread, head_rotation_sent

    mini_xception = Mini_Xception().to(device)
    mini_xception.eval()
    checkpoint = torch.load(args.pretrained, map_location=device)
    mini_xception.load_state_dict(checkpoint['mini_xception'])

    face_alignment = FaceAlignment()
    face_detector = HaarCascadeDetector('face_detector') if args.haar else
DnnDetector('face_detector')

    video = cv2.VideoCapture(0)

    stable_emotion = None
    emotion_counter = 0
    triggered = False
    STABLE_THRESHOLD = 5
    cooldown_time = 10
    last_trigger_time = 0

    t1 = 0
    ser.write(b'4')
    sts = 1
```

```
while video.isOpened():
    ret, frame = video.read()
    if not ret:
        break

    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    subjects = detector_dlib(gray, 0)

    # -------- Eye + Head Detection -------- #
    for subject in subjects:
        shape = predictor(gray, subject)
        shape = face_utils.shape_to_np(shape)

        # ---- HEAD ROTATION ----
        pitch, yaw, roll = get_head_pose(shape, frame.shape)

        cv2.putText(frame, f"Yaw: {round(yaw,1)}", (10,60),
                    cv2.FONT_HERSHEY_SIMPLEX, 0.6, (255,255,0), 2)

        if abs(yaw) > HEAD_ROTATION_THRESHOLD:
            cv2.putText(frame, "HEAD ROTATION ALERT!", (10,90),
                        cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255),2)

            if ser:
                try:
                    if not head_rotation_sent:
                        ser.write(b'1')
                        head_rotation_sent = True
                except:
                    pass
```

```
else:

            head_rotation_sent = False


# ---- EYE BLINK ----
leftEye = shape[lStart:lEnd]
rightEye = shape[rStart:rEnd]

leftEAR = eye_aspect_ratio(leftEye)
rightEAR = eye_aspect_ratio(rightEye)
ear = (leftEAR + rightEAR) / 2.0

leftEyeHull = cv2.convexHull(leftEye)
rightEyeHull = cv2.convexHull(rightEye)
cv2.drawContours(frame, [leftEyeHull], -1, (0,255,0),1)
cv2.drawContours(frame, [rightEyeHull], -1, (0,255,0),1)

if ear < EYE_THRESH:
    eye_counter += 1
    if eye_counter >= EYE_CONSEC_FRAMES:
        cv2.putText(frame, "DROWSINESS ALERT!", (10,30),
                cv2.FONT_HERSHEY_SIMPLEX, 0.7, (0,0,255),2)
        if ser:
            try:
                if(sts==1):
                    sts=0
                    ser.write(b'5')
            except:
                pass
```

```
        else:
            eye_counter = 0
            if ser:
                try:
                    if(sts==0):
                        sts=1
                        ser.write(b'4')
                except:
                    pass

    # -------- Emotion Detection -------- #
    faces = face_detector.detect_faces(frame)
    for face in faces:
        (x, y, w, h) = face
        input_face = face_alignment.frontalize_face(face, frame)
        input_face = cv2.resize(input_face, (48,48))
        input_face = histogram_equalization(input_face)

        input_face = transforms.ToTensor()(input_face).to(device)
        input_face = torch.unsqueeze(input_face, 0)

        with torch.no_grad():
            emotion_pred = mini_xception(input_face)
            emotion_index = torch.argmax(emotion_pred).item()
            emotion = get_label_emotion(emotion_index)

        if emotion == stable_emotion:
            emotion_counter += 1
        else:
            stable_emotion = emotion
```

```
            emotion_counter = 1
            triggered = False

        if (emotion_counter >= STABLE_THRESHOLD and not triggered and
            time.time() - last_trigger_time > cooldown_time):
            if voice_thread is None:
                voice_thread=threading.Thread(target=voice_interaction,
args=(stable_emotion,))
                voice_thread.start()
            triggered = True
            last_trigger_time = time.time()

        cv2.rectangle(frame, (x,y), (x+w,y+h), (255,0,0),2)
        cv2.putText(frame, emotion, (x,y-10),
                cv2.FONT_HERSHEY_SIMPLEX,0.6,(0,255,255),2)

    t2 = time.time()
    fps = round(1/(t2-t1+1e-5))
    t1 = t2
    cv2.putText(frame, f"FPS: {fps}", (10,25),
            cv2.FONT_HERSHEY_SIMPLEX,0.7,(0,255,0),2)

    cv2.imshow("Emotion + Eye Blink + Head Rotation AI Assistant", frame)
    if cv2.waitKey(1) & 0xFF == 27:
        break

video.release()
cv2.destroyAllWindows()
if ser:
    ser.close()
```

```
# ---------------- ENTRY ---------------- #
if __name__ == "__main__":
    parser = argparse.ArgumentParser()
    parser.add_argument('--haar', action='store_true')
    parser.add_argument('--pretrained', type=str,
                        default='checkpoint/model_weights/weights_epoch_75.pth.tar')
    args = parser.parse_args()
    main(args)
```

4.2 Hardware Implementation

4.2.1 Objective 1 Implementation

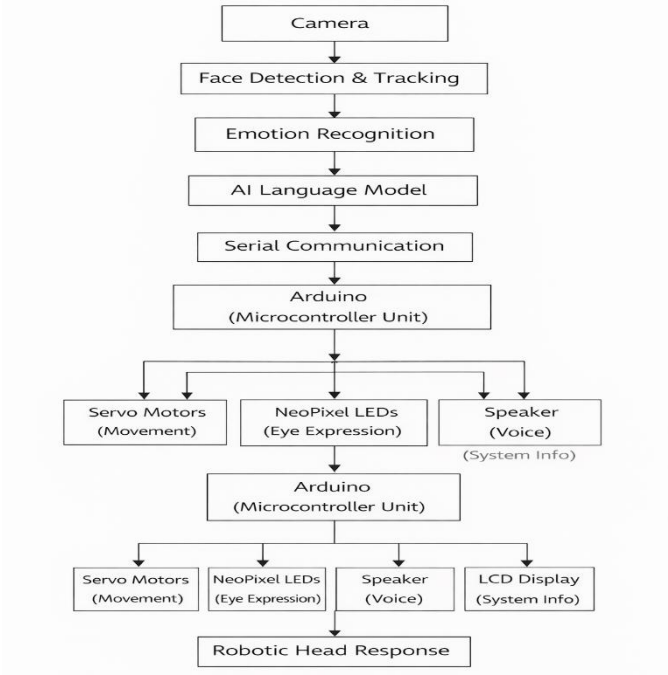


Fig 4.1 Flow Diagram of First Objective

The first objective of the proposed system is to develop a robotic head capable of detecting human facial expressions and responding with appropriate movements, visual expressions, and voice interaction.

The system uses a camera to capture the user's facial image in real time. The captured image is processed using computer vision techniques to detect the face and analyze facial expressions. The emotion recognition model identifies the user's emotion using deep learning techniques.Once the emotion is detected, the system sends commands to the Arduino Uno through serial communication. The Arduino then controls different hardware components to generate responses.Servo motors are activated to perform head and mouth movements, creating a realistic robotic interaction. NeoPixel LEDs are used as robotic eyes to visually represent emotional states through different colors or lighting patterns. The system also generates speech responses using speakers through a text-to-speech module. Additionally, the LCD display shows system messages or status information.

This integrated hardware system enables the robot to interact with users by recognizing

emotions and responding through movement, lights, and voice.

4.2.2 Objective 2 implementation

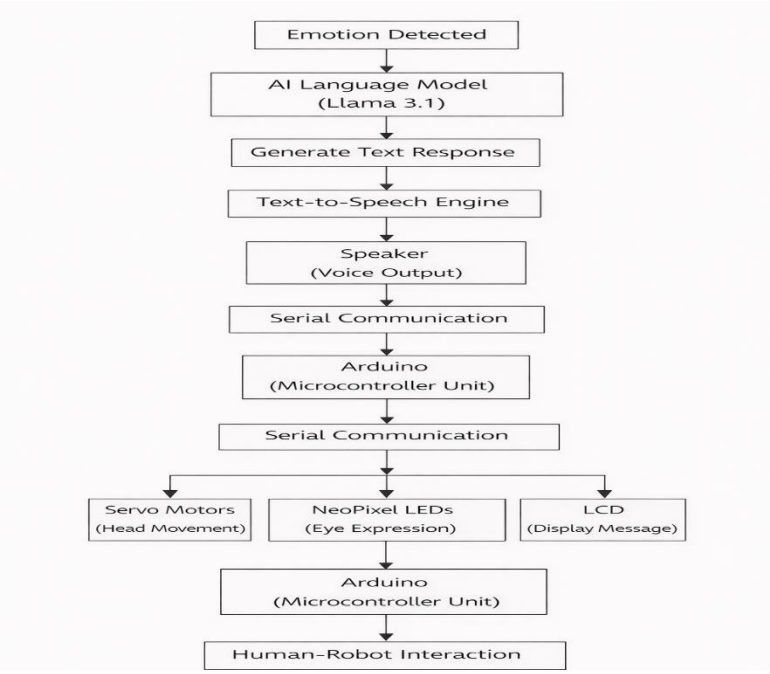


Fig 4.2 Flow Diagram of Second Objective

The second objective of the proposed system is to enable intelligent human-robot interaction through speech generation and emotional response. The system processes the detected emotion and generates an appropriate conversational reply using an AI-based language model.

After the emotion is identified by the deep learning model, the response is generated using the Llama 3.1. This model analyzes the detected emotion and produces a suitable text response to interact with the user.The generated text response is then converted into speech using a Text-to-Speech (TTS) engine. The audio output is played through the speakers connected to the robotic system, allowing the robot to communicate with the user. At the same time, the system sends signals to the Arduino Uno which controls servo motors and LED expressions. The servo motors create head or mouth movements while the NeoPixel LEDs display emotional expressions through different colors. This integration of artificial intelligence, speech synthesis, and embedded hardware enables the robotic head to interact naturally with humans and respond to emotional cues effectively.

CHAPTER-5

TESTING

5.1 Testing Approach

Testing plays an important role in ensuring that the proposed robotic system performs accurately and efficiently. The testing of the system is carried out in two stages: **software testing** and **hardware testing**.

Software testing is performed using the **Arduino IDE** and Python programming environment to verify that the emotion recognition model, AI response generation, and communication between software and hardware work correctly.

Hardware testing involves checking the functionality of physical components such as servo motors, NeoPixel LEDs, LCD display, and speakers connected to the **Arduino Uno**. This testing ensures that all components operate correctly and respond to commands sent by the software system.

Through these testing stages, the system performance, reliability, and interaction capabilities are verified to ensure that the robotic head functions as expected.

5.2 Features to be tested

After assembling the entire robotic system, several key features are tested to ensure proper system functionality.

• Verify that the camera correctly captures facial images and sends them to the emotion recognition module.

• Ensure that the deep learning model correctly identifies different human emotions such as happiness, sadness, or surprise.

• Check whether the Arduino microcontroller receives commands correctly through serial communication.

• Verify that servo motors perform proper head or mouth movements based on detected emotions.

• Ensure that NeoPixel LEDs change color or brightness to represent different emotional states.

• Check that the speakers correctly produce voice responses using the text-to-speech engine.

• Verify that the LCD display shows appropriate system messages and interaction information.

5.3 Testing tools and environment

The following tools and environment are used for testing the robotic system:

Arduino IDE:

Used to upload, debug, and test the Arduino code responsible for controlling servo motors, LEDs, LCD display, and other hardware components.

Python Programming Environment:

Used to execute the emotion recognition model, process camera input, and generate AI-based responses.

Computer System with Camera:

Used to capture facial expressions and run the software components required for emotion recognition.

Speakers and Display Devices:

Used to verify the audio and visual responses generated by the robotic system.

5.4 Test cases

5.4.1 Inputs

This project requires the following inputs for testing:

1. **Power Supply:** A regulated power supply is provided to the Arduino and connected hardware components to ensure proper system operation.
2. **Facial Image Input:** The camera captures facial expressions of the user which serve as input for the emotion recognition model.
3. **System Commands:** Commands generated by the AI system are sent to the Arduino to control servo motors, LED expressions, and voice responses.

5.4.2 Expected Output

The following outputs are expected from the proposed robotic system:

• Accurate detection and classification of human facial emotions in real time.

• Appropriate robotic head movements based on the detected emotion.

• Visual expression of emotions through NeoPixel LED eye lighting.

• Generation of voice responses using the text-to-speech system.

• Display of interaction messages and system status on the LCD display.

5.4.3 Testing Procedure

The following steps are followed to test the proposed robotic system:

• Connect the hardware components according to the circuit diagram of the robotic head system.

• Provide power supply to the Arduino microcontroller and connected devices.

• Run the Python-based emotion recognition program on the computer system.

• Capture facial expressions using the camera.

• Verify that the emotion recognition model correctly identifies the emotion.

• Observe the corresponding robotic responses such as servo movement, LED eye expressions, and voice output.

• Confirm that the LCD display shows the correct system information.

CHAPTER-6

RESULTS

The developed humanoid robotic head system was tested to evaluate its ability to recognize facial expressions, detect head movements, and interact with users in real time. The system integrates computer vision, deep learning, and hardware control to process live webcam input and generate appropriate responses through the robotic head and display modules. Experimental results demonstrate that the system successfully detects facial emotions, tracks head orientation, and communicates system status through the LCD interface while controlling hardware components such as servos and LED indicators. The following figures illustrate different operational stages of the system including emotion recognition, head rotation detection, hardware communication, and full system integration.

Fig 6.1: LCD Display Showing System Status

This figure shows the 16×2 LCD module connected to the Arduino controller. The display indicates the current system state by showing the message “TALKING”, confirming that the robotic assistant is actively responding to the user through voice output and internal processing.

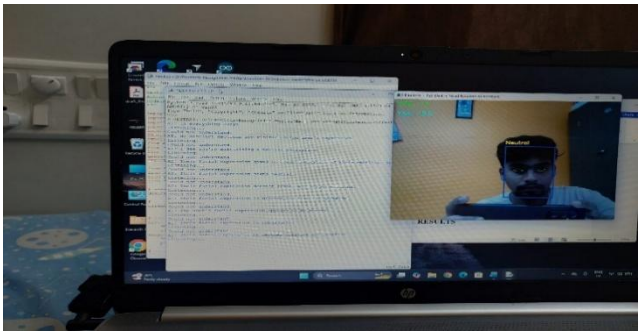


Fig 6.2: Real-Time Facial Emotion Detection

This image illustrates the real-time emotion recognition process using the webcam. The system detects the user's face and classifies the facial expression as **Neutral**, while the Python interface displays system logs and responses during interaction.

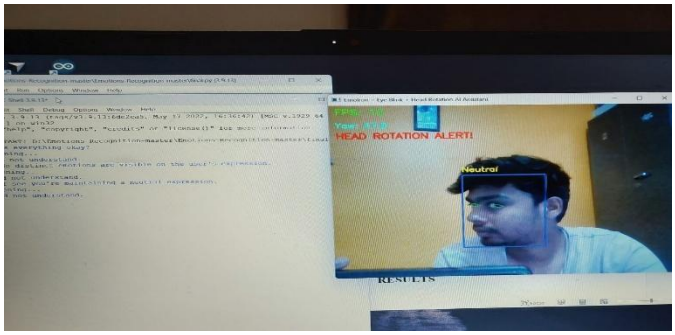


Fig 6.3: Head Rotation Detection Alert

This figure demonstrates the system's ability to detect head movement. When the user turns their head beyond a predefined threshold, the system generates a Head Rotation Alert, indicating that the user's face orientation has changed significantly.

Fig 6.4: Complete System Integration.

This image shows the fully integrated setup of the humanoid robotic head connected to the laptop and Arduino controller. The robotic head, LCD display, and computer vision interface work together to enable emotion recognition, user interaction.

CHAPTER-7

CONCLUSION

The Expressive Humanoid Robotic Head project demonstrates how simple electronic components and embedded programming can be combined to simulate basic human-like expressions and interactions. The system integrates servo motors, NeoPixel LEDs, an LCD display, and a microcontroller to produce coordinated movements and visual feedback that resemble facial gestures and head motions. Servo motors control the neck rotation and mouth movement to imitate talking, while NeoPixel LEDs act as expressive eyes and the LCD screen displays the robot's current status. Serial commands allow users to trigger different actions such as head movement, talking motion, and eye lighting, enabling interactive control of the robotic head. Smooth servo movements are implemented to make the actions appear more natural and realistic, which is important for improving human–robot interaction. Although the current system focuses on basic expressions and functions, it provides a strong foundation for future improvements such as adding sensors, voice recognition, artificial intelligence, and more advanced facial mechanisms. With these enhancements, the humanoid robotic head could be used in applications like educational robotics, assistive technologies, research in human–robot interaction, and entertainment systems. Overall, the project highlights the potential of combining robotics and embedded systems to create interactive machines capable of expressing simple emotions and communicating more effectively with humans.

REFERENCES

[1] J. Urakami and K. Seaborn, "Nonverbal cues in human-robot interaction: A communication studies perspective," *ACM Trans. Hum. Robot Interact.*, Dec. 2022.

[2] M. Salem and K. Dautenhahn, "23 social signal processing in social robotics," *Social signal processing*, p. 317, 2017.

[3] S. Saunderson and G. Nejat, "How robots influence humans: A survey of nonverbal communication in social human–robot interaction," *International Journal of Social Robotics*, vol. 11, no. 4, pp. 575–608, 2019.

[4] U. Zabala, I. Rodriguez, J. M. Martínez-Otzeta, and E. Lazkano, "Expressing robot personality through talking body language," *Appl. Sci. (Basel)*, vol. 11, no. 10, p. 4639, May 2021.

[5] L. Takayama, D. Dooley, and W. Ju, "Expressing thought: improving robot readability with animation principles," in *Proceedings of the 6th international conference on Human-robot interaction*. New York, NY, USA: ACM, Mar. 2011.

[6] S. Gross, B. Krenn, and M. Scheutz, "The reliability of non-verbal cues for situated reference resolution and their interplay with language: implications for human robot interaction," in *Proceedings of the 19th ACM International Conference on Multimodal Interaction*. New York, NY, USA: ACM, Nov. 2017.

[7] J. Gray, G. Hoffman, S. O. Adalgeirsson, M. Berlin, and C. Breazeal, "Expressive, interactive robots: Tools, techniques, and insights based on collaborations," in *HRI 2010 Workshop: What do collaborations with the arts have to say about HRI*, 2010, pp. 21–28.

[8] T. B. Brown, "Language models are few-shot learners," *arXiv preprint arXiv:2005.14165*, 2020.

[9] M. Salem, K. Rohlfing, S. Kopp, and F. Joublin, "A friendly gesture: Investigating the effect of multimodal robot behavior in human-robot interaction," in *2011 ro-man*. IEEE, 2011, pp. 247–252.

Publications Related to Project

Fwd: Provisional Acceptance of Your Paper – ICEISD 2026 External Inbox

saikumar Sk 2:57 pm
to me

---------- Forwarded message ---------
From: **ICEISD-2026 CBIT** <iceisd2026@cbit.edu.in>
Date: Wed, Apr 1, 2026, 10:32 PM
Subject: Provisional Acceptance of Your Paper – ICEISD 2026
To: <kurbasai15@gmail.com>

Dear Author,

It is our pleasure to inform you that your paper titled "Emotion-Sensitive Vision-Based Robotic Head for Natural Human–Robot Interaction" has been provisionally accepted for presentation at the International Conference on Engineering Innovations for Sustainable Development (ICEISD 2026). The conference is scheduled to be held on 10–11 April 2026 and is organized by Chaitanya Bharathi Institute of Technology (CBIT), Proddatur.

After the conference, your paper will be considered for publication as a book chapter in the Wiley Online Library, and indexed in Scopus. You are requested to complete the conference registration process by filling out the attached Google Form. Kindly note that the final acceptance of your paper is subject to successful completion of the registration process. Kindly complete your registration within **two days** using the link provided below to confirm your participation.

Registration link: https://forms.gle/u7JPJzoQmxUEc19z8

Thank you, and we look forward to your participation in ICEISD 2026.

Warm regards,
Conference Organizing Committee
ICEISD 2026
CBIT, Proddatur

Project CO-PO Mapping

SDG	Course Outcomes (COs)	Program Outcomes (POs)	Justification
SDG 9: Industry, Innovation & Infrastructure	CO1: Apply AI/ML concepts CO2: Develop real-time systems CO3: Use modern tools (OpenCV, PyTorch, Arduino)	PO1: Engineering knowledge PO3: Design solutions PO5: Modern tool usage	Proposed system uses AI-based emotion recognition and real-time processing with computer vision and embedded systems, contributing to technological innovation and smart infrastructure.
SDG 4: Quality Education	CO1: Understand AI & Robotics fundamentals CO3: Implement practical systems	PO2: Problem analysis PO10: Communication PO12: Life-long learning	The project acts as an interactive learning tool, improving student engagement and understanding of AI, robotics, and human-machine interaction.
SDG 10: Reduced Inequalities	CO4:Analyze user behavior CO5: Design user-friendly system	CO4:Analyze user behavior CO5: Design user-friendly system	The project supports assistive technology for elderly and differently-abled individuals, enabling inclusive human-robot interaction.
SDG 8: Decent Work & Economic Growth	CO2:Develop intelligent systems CO5:Project development skills	PO9:Teamwork PO11:Project management	The project enhances employability skills in AI, robotics, and automation, contributing to economic growth and job opportunities.

शिक्षा मंत्रालय
MINISTRY OF EDUCATION

NATIONAL INTERNSHIP PORTAL

EduSkills®
Nation Building Through Skills

AICTE - EduSkills 2026 VIRTUAL INTERNSHIP

Certificate of Virtual Internship

This is to certify that

PABBATHI SAISREE

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-weeks

Prompt Engineering for AI Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Supported by EduSkills ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education

Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 4607ab2c55d20f770c68

Student ID: STU658d8eb9076ae1703775929

INTERNSHIP GRADE O

GRADE- O (Outstanding): 90-100 | E (Excellent): 80-89 | A (Very Good): 70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

शिक्षा मंत्रालय
MINISTRY OF EDUCATION

NATIONAL INTERNSHIP PORTAL

EduSkills®
Nation Building Through Skills

AICTE - EduSkills 2026 VIRTUAL INTERNSHIP

Certificate of Virtual Internship

This is to certify that

GUTIPALLI SRAVANTH

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-weeks

Prompt Engineering for AI Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Supported by EduSkills ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education

Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 4678b0f943f0ea810337

Student ID: STU6688022866b241720189480

INTERNSHIP GRADE O

GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

शिक्षा मंत्रालय
MINISTRY OF EDUCATION

NATIONAL INTERNSHIP PORTAL

EduSkills®
Nation Building Through Skills

AICTE · EduSkills 2026 VIRTUAL INTERNSHIP

Certificate of Virtual Internship

This is to certify that

MOLLA SHAHID Basha

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-weeks

Java Full Stack Developer Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Supported by EduSkills ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education

Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 4d9f93c5dbf651614f00

Student ID: STU65892f8299cfa1703489410

INTERNSHIP GRADE E

GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

शिक्षा मंत्रालय
MINISTRY OF EDUCATION

NATIONAL INTERNSHIP PORTAL

EduSkills®
Nation Building Through Skills

AICTE · EduSkills 2026 VIRTUAL INTERNSHIP

Certificate of Virtual Internship

This is to certify that

Sai KUMAR

Srinivasa Ramanujan Institute of Technology

has successfully completed the 10-weeks

Prompt Engineering for AI Virtual Internship

During January - March 2026

May this Internship learning propel you toward a bright and successful career.

Supported by EduSkills ACADEMY

Dr. Buddha Chandrasekhar
Chief Coordinating Officer (CCO)
AICTE, Ministry of Education

Shubhajit Jagadev
Chief Executive Officer (CEO)
EduSkills

Certificate ID : 43dea05d843768d05f80

Student ID: STU6596c1ba93cd91704378810

INTERNSHIP GRADE O

GRADE- O (Outstanding):90-100 | E (Excellent):80-89 | A (Very Good):70-79 | B (Good): 60-69 | C (Fair): 50-59 | D (Average): 40-49 | P (Pass): 30-39 | F (Fail): Below 30

DETAILS OF PROJECT TEAM

	Mr. A. Kiran Kumar Yadav M.Tech., (Ph.D) Associate Professor Computer Science & Engineering (AI & ML) Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh kirankumara.cse@srit.ac.in
	P. Saisree 224G1A3392 Computer Science & Engineering (AI & ML) Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 224g1a3392@srit.ac.in
	G. Sravanth 224G1A33A5 Computer Science & Engineering (AI & ML) Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 224g1a33A5@srit.ac.in
	M. Shahid Basha 224G1A33A0 Computer Science & Engineering (AI & ML) Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 224g1a33A0@srit.ac.in
	B. Sai Kumar 234G5A3307 Computer Science & Engineering (AI & ML) Srinivasa Ramanujan Institute of Technology, Anantapur, Andhra Pradesh 234g5a3307@srit.ac.in